北京理工大學

本科生毕业设计（论文）外文翻译

外文原文题目： Hopter: a Safe, Robust, and Responsive Embedded Operating System

中文翻译题目： Hopter：一个安全、健壮且高响应能力的嵌入式操作系统

Alien 内核的 x86_64 移植与 vDSO 引入

Alien Kernel: x86_64 Porting & vDSO Integration

学　　院：计算机学院

专　　业：计算机科学与技术（徐特立英才班）

班　　级：07022202

学生姓名：胡嘉晨

学　　号：1120220611

指导教师：陆慧梅

原文：

Hopter: a Safe, Robust, and Responsive Embedded Operating System

Zhiyao Ma
zhiyao.ma@yale.edu
Yale University
New Haven, CT, USA

Guojun Chen
guojun.chen@yale.edu
Yale University
New Haven, CT, USA

Zhuo Chen
zhuo.chen.zc384@yale.edu
Yale University
New Haven, CT, USA

Lin Zhong
lin.zhong@yale.edu
Yale University
New Haven, CT, USA

Abstract

Microcontroller-based embedded systems are vulnerable to memory safety errors and must be robust and responsive because they are often used in unmanned and mission-critical scenarios. The Rust programming language offers an appealing compile-time solution for memory safety but leaves stack overflows unresolved and foils zero-latency interrupt handling. We present Hopter, a Rust-based embedded operating system (OS) that provides memory safety, system robustness, and interrupt responsiveness to embedded systems while requiring minimal application cooperation. Hopter executes Rust code under a novel finite-stack semantics that converts stack overflows into Rust panics, enabling recovery from fatal errors through stack unwinding and restart. Hopter also employs a novel mechanism called soft-locks so that the OS never disables interrupts. We compare Hopter with other well-known embedded OSes using controlled workloads and report our experience using Hopter to develop a flight control system for a miniature drone and a gateway system for Internet of Things (IoT). We demonstrate that Hopter is well-suited for resource-constrained microcontrollers and supports error recovery for real-time workloads.

CCS Concepts

• **Computer systems organization** → **Embedded systems**; **Reliability**; **Real-time operating systems**.

Keywords

Embedded System, Operating System, Rust, Memory Safety, Robustness, Interrupt Responsiveness

ACM Reference Format:
Zhiyao Ma, Guojun Chen, Zhuo Chen, and Lin Zhong. 2025. Hopter: a Safe, Robust, and Responsive Embedded Operating System. In *The 23rd Annual International Conference on Mobile Systems, Applications and Services (MobiSys '25), June 23–27, 2025, Anaheim, CA, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3711875.3729149

1 Introduction

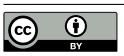
Embedded systems have seen increasing adoption in the past decade, and their number is predicted to increase even more in the coming decades [37–39]. Due to resource constraints, many embedded systems employ microcontrollers without a memory management unit (MMU) and as a result, do not enjoy the safety associated with virtual memory. To make things worse, C, an unsafe language, has been the most common language for programming such microcontroller-based embedded systems. Consequently, these systems can suffer from memory-related security vulnerabilities that are difficult to discover, as is evident from those reported for FreeRTOS [17, 18] a decade after its release. With microcontroller-based embedded systems increasingly used in mission-critical applications, it is imperative to enhance their memory safety and system robustness without demanding more resources or sacrificing system responsiveness.

The Rust programming language offers an attractive alternative to C. It guarantees memory safety primarily through compile-time checks, incurring little runtime overhead, and has already seen adoption in operating system (OS) development [8, 13, 14, 26, 50, 56], including embedded systems [41, 55, 63].

However, the use of Rust on microcontroller-based embedded systems faces its own challenges. First, memory safety errors linger on even with safe Rust, because its semantics assumes an infinite size of function call stacks, which is especially problematic for microcontroller-based embedded systems where there is no virtual memory and only 10s to 100s KiB of SRAM are usually available. Vulnerability reports have shown the existence of stack overflows in Rust libraries, including those intended for embedded use [15, 16, 19, 22–24]. Second, Rust's built-in exception handling mechanism, i.e., panics, requires a stack unwinder, which is usually unavailable on microcontrollers. Without a stack unwinder, panics lead to hang or reset of the application [41] or the whole system [11, 12]. Finally, language restrictions of Rust make it difficult to implement zero-latency interrupt handling, where the OS never disables interrupts to ensure timely response to events. Known solutions [47, 48, 57, 58] are infeasible with safe Rust because they struggle to pass the compile-time check. §2 elaborates on the challenges.

In this paper, we seek to answer the following question: Can we bring memory safety, system robustness, and interrupt responsiveness to a Rust-based embedded system while requiring minimal application cooperation? We present a positive answer with the Hopter embedded OS (§3). Hopter features a co-design between the OS and the implementation language to complete memory safety and achieve fatal error resilience. Hopter also brings zero-latency interrupt handling to a Rust-based system running threaded tasks, using a novel mechanism called soft-locks.

Hopter augments Rust with finite-stack semantics (FS-semantics) to achieve stack memory safety and overflow resilience (§4.1). FS-semantics treats each function call as a potential panic site. Before executing the function body, a prologue checks the remaining stack space and will raise a panic if insufficient. In case a drop handler



MobiSys '25, Anaheim, CA, USA

ACM ISBN 979-8-4007-1453-5/2025/06
https://doi.org/10.1145/3711875.3729149

or its callee overflows the stack, Hopter will temporarily extend the stack to finish the drop handler [35, 45, 67] and raise a panic afterward. This effectively unifies stack overflows with other fatal errors using Rust panics.

Hopter then reclaims the resources upon panics utilizing a customized stack unwinder, which allows Hopter to automatically restart failed tasks (§4.2). When possible, Hopter performs a concurrent restart to expedite recovery, where the restarted task instance runs concurrently with the unwinding procedure of the failed one.

Soft-locks enable zero-latency interrupt handling by serializing mutations for each OS object, avoiding the global serialization queue and therefore unsafe Rust (§4.3). When a task or interrupt handler acquires a soft-lock that is not under contention, it gains full access to the protected object. Otherwise, it gains partial access to record its intended operations on the object and the soft-lock will commit them after contention.

We implement and open-source Hopter as a Rust library crate for Arm Cortex-M0 and Cortex-M4 architectures (§5). Hopter requires a customized compiler to compile the system, but the Rust syntax remains the same and the semantics compatible. Hopter also supports unmodified third-party hardware abstraction layer (HAL) crates, allowing application programmers to tap the growing Rust ecosystem. We evaluate Hopter by comparing it with two other embedded OSes, namely FreeRTOS and Tock (§6). Hopter has lower interrupt handling latency than FreeRTOS and is safer and more robust. Hopter requires less hardware resources than Tock and supports microcontrollers that lack a memory protection unit (MPU). We develop a flight control system for the Crazyflie 2.1 miniature drone and a gateway system for Internet of Things (IoT) devices. With the flight control system, Hopter exhibits a 56% increase in flash usage and proportionally 15% higher CPU load incurred together by FS-semantics, the unwinding mechanism, and soft-locks. The gateway system showcases Hopter's robustness against runtime errors, even on a Cortex-M0 CPU without MPU.

We make the following contributions:

- Finite-stack semantics for Rust to guarantee stack memory safety and overflow resilience, implemented via compile-time instrumentation and OS support.
- Soft-locks, a novel synchronization primitive which enables zero-latency interrupt handling on Rust-based systems with threaded tasks.
- Open-source implementation of the Hopter embedded OS [25], which integrates FS-semantics and soft-locks to deliver memory safety, failure resilience, and responsiveness, while requiring minimal application cooperation.
- Evaluation of the code size and performance overhead incurred by FS-semantics, the unwinding mechanism, and soft-locks, showing Hopter's suitability for resource-constrained microcontrollers and real-time workloads.

2 Background

Microcontrollers are widely used in embedded systems, from simple home appliances [34, 49, 65] to complex automotive control systems [28, 31, 40] and industrial automation [10, 46]. Many of these systems are unmanned or mission-critical, making robustness and the ability to recover from fatal errors essential. Due to cost and energy constraints, the hardware resources available on microcontrollers are usually limited. They typically feature hundreds of kilobytes to a few megabytes of flash for code and read-only data, and tens to hundreds of kilobytes of SRAM for runtime data. Consequently, operating systems designed for microcontrollers must be lightweight.

Microcontrollers operate without virtual memory. All code runs in a single physical address space where flash storage, SRAM, and peripheral registers are mapped. The CPU typically executes instructions directly from the byte-addressable flash, called execute in place (XIP), while function call stacks and mutable data reside in SRAM. The code on microcontrollers usually has unrestricted access to the entire address space, making the system prone to memory safety errors like buffer overflows and invalid pointer accesses, which can lead to system instability or security vulnerabilities. Due to the lack of isolation between application tasks and the operating system, attackers can exploit memory safety vulnerabilities and easily gain full control of the entire system.

Hardware-based memory protection, e.g., memory protection unit (MPU) [2] on Arm and physical memory protection (PMP) [54] on RISC-V, is available on high-end microcontrollers and has been actively exploited by some embedded OSes, for example, Tock [41]. Unfortunately, hardware-based protection introduces overheads in code size, memory usage, and runtime performance. MPU/PMP allows developers to define up to 16 memory regions with selective read, write, or execute permissions. However, each memory region must be aligned and sized to the nearest power of two, leading to wasted flash or SRAM due to internal fragmentation. Moreover, employing an MPU or PMP requires system calls to switch privilege modes via software interrupts, and arguments need to undergo marshaling and be verified by the OS, adding performance overhead. In contrast, in systems without such protection mechanisms, system calls can be efficiently implemented as simple function calls. As a result, popular embedded OSes such as FreeRTOS [1] consider hardware-based protection optional, even if the system is written in an unsafe language like C.

Rust Programming Language is a modern systems programming language that provides memory and concurrency safety while offering direct control over hardware. It presents a promising alternative to hardware-based protection without the significant overhead incurred by managed languages. Rust has seen an increase in adoption in OS development [8, 13, 14, 26, 50, 56] and embedded systems [41, 55, 63].

Rust manages resources through its ownership model without using a garbage collector. Each value is owned by a variable, and when the variable goes out of scope, the value's *drop handler*, known as destructor function in other languages, runs to release the resource. Unlike other languages that usually make a copy or a reference when assigning a value to a new variable, Rust by default *moves* the ownership of the value from the old variable to the new one. The old variable becomes inaccessible after the move. A function call in Rust also by default moves the ownership of argument values to the callee, which is then responsible for invoking the drop handlers of these values. Consequently, the call stack is likely to reach its maximum depth while calling drop handlers, a phenomenon confirmed by our flight control system (§6.3).

Rust enforces the ownership model and analyzes reference lifetimes at compile-time to ensure memory safety, unless opted out with the `unsafe` keyword. Importantly, the compile-time check rejects some common code patterns. For instance, a struct in Rust may contain a reference only when the reference's lifetime in the syntactical scope outlives the struct's. Such restrictions preclude known implementations [47, 48, 57, 58] for zero-latency interrupt handling, where a global serialization queue is necessary to store pending operations containing references to various OS objects. To solve this problem, Hopter introduces a novel mechanism called soft-locks (§4.3) to achieve zero-latency interrupt handling.

Rust complements its compile-time check with a language exception mechanism called *panic* to forestall memory errors that are detectable only at runtime, such as out-of-bounds array access. Failed assertions through `assert!` or `unwrap` also lead to panics. A panic terminates the normal execution flow of a Rust thread. On resourceful systems such as personal computers, a panic is usually followed by a stack unwinding procedure that iterates through the function frames in the call stack and invokes the drop handlers of live objects to reclaim resources. This allows subsequent recovery from the error without restarting the entire system. However, embedded Rust systems usually lack a stack unwinder [11, 12, 41] and as a result, a panic will hang or reset the system. Worse, assertions are common in embedded Rust code [30, 41]. For system robustness, Hopter incorporates a stack unwinder optimized for microcontrollers (§4.2).

A loophole of Rust's memory safety guarantee is that function call stacks can still overflow. Rust semantics assumes an infinite stack space for each running thread, which is particularly unrealistic on microcontrollers. Common approaches to detecting stack overflows, including stack protectors (canaries) [3, 32] and periodic stack pointer inspection [4], are belated efforts. By the time of detection, the system memory is already corrupted. Prior Rust-based embedded systems either use MPU/PMP to forestall stack overflows [41], accepting the associated overhead, or employ only a single down-growing stack placed at the lower boundary of the SRAM region [11, 12], which restricts scheduling patterns. Hopter addresses this problem by running Rust code with finite-stack semantics (§4.1), which not only prevents stack overflows but also converts such errors into Rust panics to allow a unified recovery procedure through unwinding and restarting.

3 System Development with Hopter

Before we present the design (§4) and implementation (§5) of Hopter, we describe how a system developer may use it to develop an embedded system with application code.

3.1 Development Model

Hopter is open-source and a system developer receives it as a Rust library crate. Since Hopter supports threaded tasks and forsakes hardware-based protection, it can interoperate with third-party HAL libraries [29, 30], which significantly lowers the development effort. The developer must write application code in Rust and use a customized Rust compiler supplied by Hopter to build their system that uses Hopter as a dependency. Downloading and registering the customized compiler with `cargo` requires only three commands.

```rust
#[main]
fn main() {
    // Initialize resource for another task.
    // Stored behind atomic reference counting so
    // that the variable has the `Clone` trait.
    let res = Arc::new(init_resource());

    // The entry closure then also has the `Clone` trait.
    // Can spawn the task as a restartable one.
    task::build()
        .set_entry(move || another_task_main(res))
        .set_priority(8)
        .set_stack_limit(1024)
        .spawn_restartable()
        .unwrap();
}
```

Listing 1: Application code starts the execution from its main function marked by the `#[main]` attribute. Other tasks can be started dynamically through the task builder pattern. When the entry closure has the `Clone` trait, the task can be spawned as a restartable one.

The developer compiles the system with `cargo build` as usual and the Rust syntax remains the same.

Hopter expects application code to be benign, because application code may use `unsafe` Rust that potentially introduces memory errors. Hopter guarantees memory safety of the system as long as all `unsafe` code used by the application is sound. This expectation is similar to that of the popular FreeRTOS [1]; we consider it reasonable since all source code to be compiled is usually available to the system developer of a microcontroller-based embedded systems. Hopter's threat model is weaker than that of Tock [41] where application code can be malicious and the OS must isolate itself using hardware-based protection (See §2), along with its overhead.

3.2 Using Hopter Abstractions

Hopter provides three important abstractions for system developers to achieve memory safety, fault tolerance, and interrupt handling promptness. To implement application logic, a developer uses either the *restartable task* or the *fallible interrupt handler* context abstraction, which guarantees memory safety and allows recovery from fatal errors. Fallible interrupt handlers have zero-latency in response time to hardware events and can coordinate with tasks through provided *synchronization primitives*. Here we elaborate each of the three key abstractions: its benefits and how to use it. Their design and implementation details will be presented in §4 and §5, respectively.

Restartable Tasks: The tasks running on Hopter are thread-based and scheduled preemptively. They improve applications' resilience against fatal errors through automatic restarting. Applications on Hopter spawn tasks with the builder pattern as shown in Listing 1, providing the entry closure and specifying the attributes. The main task is an exception that starts execution with the function marked with the provided `#[main]` attribute, which usually initializes the system and spawns other tasks. To enable automatic restarting, the application task needs only ensure that its entry closure implements the `Clone` trait and is started with the `spawn_restartable()` method. The `Clone` trait allows Hopter to duplicate the entry environment

```rust
// Initialized to `Some(_)` after booting.
// Spin lock used to satisfy interior mutability rule.
static TIMER: Spin<Option<CounterUs<TIM2>>>
    = Spin::new(None);
static MAILBOX: Mailbox = Mailbox::new();

// Invoked periodically by the timer IRQ.
#[handler(TIM2)]
fn tim2_handler() {
    TIMER.lock()
         .as_mut().unwrap() // Unwrap `Option`
         .wait().unwrap();  // Acknowledge IRQ
    // Resume the task.
    MAILBOX.notify_allow_isr();
}

// Spawned as a task.
fn another_task_main() {
    loop {
        MAILBOX.wait(); // Block and yield CPU
        // Do something else ...
    }
}
```

Listing 2: Synchronization between a handler and task using a mailbox. The interrupt handler is declared through the `#[handler(...)]` macro. The handler has zero-latency response time while still being able to call synchronization primitive methods. Peripheral access is provided through a third-party HAL crate [30].

necessary to restart a task. A closure has the `Clone` trait if all of the enclosed variables are `Clone`. Upon fatal errors like stack overflows, out-of-bounds array accesses, and failed assertions, restartable tasks terminate, release their resources, and automatically restart execution from beginning. Restartable tasks do not revert to an explicit checkpoint. Instead, values stored in the heap or declared with `static` persist across task restarts, while function local values are dropped. If the entry closure is not `Clone`, the `spawn()` method is still available to run the task, but the task will only terminate with resources released upon fatal errors rather than restart.

Fallible Interrupt Handlers: The interrupt handlers on Hopter are functions that respond to hardware interrupts (IRQ), sharing a single interrupt stack and always preempting tasks or lower-priority handlers. They improve system robustness by tolerating fatal errors during interrupt handling. Applications declare interrupt handler functions using the `#[handler(IRQ_NAME)]` attribute macro as shown in Listing 2. A handler terminates upon fatal errors with resources released, and will rerun if the interrupt is still pending. Hopter cannot recover from interrupt stack overflows, due to the restrictions of dynamic memory allocation within the interrupt context.

Synchronization Primitives: Hopter provides applications with synchronization primitives in Rust `struct` types to facilitate inter-context coordination. They allow synchronization between interrupt handlers and tasks without compromising the zero-latency response time to interrupts. Application code interacts with synchronization primitives by calling defined methods. Listing 2 shows an example of the synchronization between a timer interrupt handler and a task using a mailbox. The hardware timer is accessed through a third-party HAL crate [30].

4 System Design

We next present the key design aspects of Hopter to realize memory safety, fault tolerance, and interrupt responsiveness. Hopter employs a compiler-based mechanism to guarantee memory safety for restartable tasks and fallible interrupt handlers. Notably, for stack memory safety, Hopter executes Rust code with finite-stack semantics (FS-semantics), facilitated by compile-time instrumented code (§4.1). FS-semantics unifies runtime memory errors and other fatal errors as Rust panics, allowing Hopter to apply a universal recovery mechanism based on unwinding and restarting (§4.2). Moreover, Hopter overcomes expressiveness challenges associated with Rust, achieving zero-latency interrupt handling with a novel mechanism called soft-locks (§4.3) to support the implementation of synchronization primitives. Application logic remains agnostic to both the compile-time instrumentation and OS-internal implementation, allowing existing Rust libraries to be used unmodified.

4.1 Rust with Finite-stack Semantics

Hopter executes Rust code with *finite-stack* semantics (FS-semantics) to ensure stack memory safety and allow system recovery from stack overflows. FS-semantics associates each Rust thread of execution with an implicit stack size. Hopter forestalls any function call made without sufficient free stack space by raising a panic (§4.1.1). In the exceptional case where an overflow occurs during the execution of a drop handler, Hopter temporarily extends the stack to finish the drop handler and raises the panic afterward (§4.1.2). Hopter takes two steps to achieve FS-semantics:

4.1.1 Forestalling Stack Overflows. Hopter detects an impending stack overflow by examining the free stack space before executing a function body. This is achieved by a prologue of instructions emitted by the compiler before the function body that allocates a stack frame. The prologue computes the free stack size as the difference between the current stack top indicated by the stack pointer and the stack region boundary stored in a task-local variable `BOUNDARY`. If there is insufficient free space, it traps into the OS for further diagnosis. The execution of the function prologue requires at most 8 bytes of reserved stack space on Arm.

Hopter raises a panic to the task experiencing a stack overflow. The panic initiates stack unwinding that reclaims the resources from the failed task to allow restarting it without leaked resources or deadlock. In the common case, Hopter raises a panic immediately upon detecting an imminent overflow by the prologue. After trapping, Hopter sets the program counter of the task to the stack unwinder entry to start the unwinding.

In the rare case where the stack overflows during the execution of a drop handler, Hopter extends the stack to finish running the drop logic before raising a panic. This is because stack unwinding must not start inside a drop handler, as doing so would skip some code responsible for releasing resources. More precisely, Hopter extends the stack and defers the panic if the function overflowing the stack meets either of the following two criteria: (1) Is a drop handler, or (2) Is called directly or indirectly by a drop handler.

Two separate mechanisms are required to check these criteria. The first criterion is addressed by the function prologue, which passes to Hopter whether the offending function is a drop handler.

Checking for the second criterion requires drop handler instrumentation. Whenever a drop handler starts executing, it sets the `IN_DROP` task-local variable to `true`. The drop handler resets the variable to `false` after finishing its logic. In the case of nested drop handler invocations that occur with nested struct types, only the outermost one clears the flag before returning. Hopter can determine if the second criterion is met by observing the value of the `IN_DROP` flag. Note that the function prologue is still applied on top of the instrumentation for drop handlers.

Hopter defers raising the panic if either of the criteria holds upon overflow. It sets the `OVERFLOWED` task-local variable to `true` and extends the stack, rather than raising a panic immediately. After finishing the drop logic, the outermost drop function will check the `OVERFLOWED` flag and raise the deferred panic if necessary.

Since any function can overflow the stack and initiate stack unwinding under FS-semantics, this conflicts with some compiler optimizations. The LLVM compiler backend infers the `nounwind` attribute for functions and simplifies generated code based on it. LLVM marks a function as `nounwind` if it satisfies the following two conditions [42, 43]: (1) The function body contains no side-effect instruction. (2) The function makes calls to only `nounwind` functions or is a leaf function. LLVM then simplifies the code by omitting the landing pads and unwind table entries if a call is made to a `nounwind` function. However, if such a function call overflows the stack, the stack unwinding will fail, causing a hang or system reset. Therefore, Hopter disables these compiler optimizations for correctness.

4.1.2 Extending Stacks. Hopter leverages segmented stacks [35, 45, 67] to extend stacks on systems without virtual memory. A segmented stack is a linked list of non-contiguous memory chunks called stacklets. It dynamically allocates a new stacklet upon function calls if the remaining stack space is insufficient, and frees it upon function return.

The function prologue (§4.1.1) facilitates stack extension by providing Hopter with the requested stack frame size and stack-passed argument size. These two numbers are constants known to the compiler during compilation and are embedded in the instruction sequence as literals to avoid using extra registers. Hopter subsequently fetches the values by following the program counter prior to the trap and then allocates a stacklet with a size no less than the sum of the two numbers. If stack-passed arguments exist, Hopter will also copy them to the newly allocated stacklet so that they are adjacent to the callee's stack frame. Finally, Hopter changes the task's stack to the new stacklet and resumes the task so that the latter continues to execute the function body.

We note that the stack extension mechanism requires software-based overflow detection. This is because the stack change must happen before executing the function body. In contrast, hardware-based protection typically detects the overflow after the function body has started execution and accessed an invalid memory address. Transferring execution to a new stacklet midway is very difficult because copying a stack frame breaks internal references.

4.2 Recovery from Panics

Hopter supports recovery from Rust panics in both tasks and interrupt handlers. A panic occurs due to an out-of-bounds array access, a failed assertion, or a stack overflow under FS-semantics. When a task or an interrupt handler panics, Hopter terminates its execution and reclaims allocated resources. The panic is isolated within the context where it is raised and does not prevent the system from continuing execution. If a task's entry closure implements the `Clone` trait, Hopter can automatically restart it upon a panic. Thus, interrupt handlers on Hopter are *fallible* and tasks *restartable*.

4.2.1 Unwinding Stacks. Hopter integrates a stack unwinder to reclaim resources upon panics, enabling graceful termination of tasks or interrupt handlers and allowing subsequent recovery. The unwinding procedure runs in the context of the panicked task or interrupt handler, during which the scheduler can continue to perform context switches and higher-priority interrupts can nest atop. Logically, the unwinder forces the immediate return of active functions when a panic occurs, starting from the top of the call stack and proceeding until the entry function of a task or interrupt handler returns. Local objects are dropped during this procedure. Mechanically, the unwinder refers to the unwind table generated by the compiler to restore registers and invoke small pieces of code called landing pads. There are two types of landing pads: `cleanup` pads, which call drop handlers, and `catch` pads, which terminate the unwinding procedure.

Hopter's stack unwinder differs from existing ones [36, 53, 64] in two ways to support segmented stacks. First, Hopter's unwinder avoids making divergent function calls that never return. Since the unwinder may be invoked to clean up a task experiencing a stack overflow, it must extend the stack to execute its own logic. If the unwinder made divergent calls, the stacklets used would not be freed, leading to memory leaks. In contrast, prior implementations typically make divergent calls to landing pads. Second, Hopter's unwinder recognizes stacklet boundaries and frees stacklets during unwinding to prevent memory leaks. This is important because a task's stack may contain multiple stacklets, e.g., when an overflowing drop handler raises a deferred panic while running on an extended stacklet.

4.2.2 Restarting Applications. With resources reclaimed by the unwinder, Hopter can recover a panicked task by restarting it from its entry closure. To enable restarting, Hopter requires the task's entry closure to implement the `Clone` trait so that it can safely duplicate the closure's enclosed environment. For interrupt handlers, Hopter performs an exception return instead of re-executing the handler after catching the panic. If the interrupt request is still pending, the handler will automatically be invoked again.

To speed up recovery from task panics, Hopter implements the *concurrent task restart* optimization proposed by [44]. This technique allows for immediate execution of a new instance of the panicked task, running concurrently with the unwinding procedure of the previous instance, thereby ensuring minimal unresponsive time. Hopter reduces the task priority of the panicked instance to the lowest, allowing unwinding to utilize idle CPU time. Rust's ownership model eliminates race conditions between the restarted instance and the one being unwound. Application programmers may opt out of concurrent restart to prevent the restarted task from observing logically inconsistent data. In this case, Hopter performs a *context reuse* optimization, reusing the task structure of the unwound task for the restarted instance.

4.3 Zero-latency Interrupt Handling

Hopter supports zero-latency interrupt handling, invoking the handler function immediately upon receiving an interrupt. This is possible because Hopter never disables interrupts. To prevent race conditions between the interrupt handler and the preempted context, Hopter adopts a novel mechanism called *soft-locks* that are amenable to Rust's compile-time check. These locks protect OS data structures by serializing mutations from concurrent contexts, allowing interrupt handlers in Hopter to interact with OS objects, e.g., using synchronization primitives to notify tasks.

4.3.1 Algorithm Description. Soft-locks eliminate race conditions caused by concurrent access from interrupt handlers. Acquiring a soft-lock yields either *full access* or *partial access* to the protected data structure. Code gets full access if the soft-lock is not already acquired or otherwise partial access. Full access allows mutations to all data structure fields, while partial access permits modifications to only a subset of fields.

An interrupt handler receives partial access to record its intended operations when a preempted context holds the full access. These operations are deferred until the handler returns and the code with full access completes its own operation. To further prevent race conditions arising from concurrent task execution, the scheduler is always suspended before acquiring a soft-lock, thus code running within task contexts always receives full access. Because interrupt handlers always preempt tasks and have strict priority, code with full access resumes execution after the handler with partial access finishes. Therefore, deferred operations can run without conflict immediately after the code with full access finishes its logic.

Soft-locks encapsulate the protected data structure and maintain two atomic boolean flags to track contention states and whether any deferred operations are pending. Listing 3 shows the pseudo-code for acquiring and releasing soft-locks. The `locked` flag indicates whether the data structure is under contention, and the `pend` flag indicates if there are deferred operations. The protected data structure must implement the `AllowDeferredOp` trait to specify which fields are accessible through the full or partial accessor and the code for running deferred operations. Upon releasing a soft-lock, pending deferred operations are checked and executed if necessary using the scoped full accessor, which makes it amenable to compile-time checks. In the actual implementation, soft-locks are released automatically through the drop functions of access guards instead of manually.

4.3.2 Use Cases in Hopter. Soft-locks protect four data structures in Hopter. Table 1 lists these data structures, along with the operations under full or partial access and the deferred operations. The wait queue provides the foundation to implement synchronization primitives like mutexes, condition variables, semaphores, and channels. The ready queue maintains ready tasks for the scheduler. The mailbox is a lightweight synchronization primitive that allows tasks to wait for a notification, optionally with a timeout. The sleep queue stores sleeping tasks waiting for virtual timers to expire. An interrupt handler may notify a task by removing it from the wait queue or mailbox, adding it to the ready queue, and optionally deleting it from the sleep queue.

```
struct SoftLock<T> where T: AllowDeferredOp {
    locked: AtomicBool, pend: AtomicBool, ...
}

fn acquire(sl: &SoftLock<T>) -> Access {
    let prev_locked = sl.locked.swap(true);
    if prev_locked { return Access::Partial; }
    else { return Access::Full; }
}

fn release(sl: &SoftLock<T>, acc: Access) {
    match acc {
        Access::Partial => { sl.pend = true; }
        Access::Full => {
            loop {
                let prev_pend = sl.pend.swap(false);
                if prev_pend { acc.deferred_op(); }
                sl.locked = false;
                if !sl.pend { break; }
                else { sl.locked = true; }
            }
        }
    }
}
```

Listing 3: Pseudo-code for the `acquire` and `release` operation on soft-locks. Protected data structures must implement the `AllowDeferredOp` trait to specify what fields are accessible through the `Full` or `Partial` accessor and what code to execute when running `deferred_op()`. Soft-locks return the `Full` accessor under no contention or otherwise `Partial`. Deferred operations are executed when releasing with the `Full` accessor. The loop avoids missing deferred operations that come after the first check of the `pend` flag.

When a protected data structure is under contention, the handler uses partial access to record its intended operations, which are deferred for later execution. For example, consider a task that blocks on a mailbox to wait for a hardware event. The task first acquires full access to the mailbox to enter the placeholder `H`. If the interrupt occurs during the modification of `H`, the handler will get partial access to the mailbox that only allows it to increment the notification counter `x`. After the handler returns, the task resumes its operation, but will then notice the incremented counter when releasing the full access. The task then decrements the counter and cancels its blocking. In a different scenario where the interrupt occurs after the task finishes its operation, the task will have already released the full access and blocked itself. The handler will acquire full access to the mailbox and unblock the task in `H`.

5 Implementation

We have implemented Hopter for Arm Cortex-M4 and Cortex-M0 architectures with approximately 15,000 lines of code. To use Hopter for a system with a microcontroller of either architecture, a developer only needs to define an interrupt vector array, because Hopter leverages existing HAL libraries for peripheral access. The implementation consists of two parts: compiler modifications and Rust code for the library crate. Additionally, we implement Hopter's features in a hierarchy so that developers can make trade-offs between their benefits and overheads.

Table 1: OS data structures protected by soft-locks. Code gets full access to the data structure to perform one of the listed operations if the soft-lock is not under contention. Code running in task context always gets full access. An interrupt handler gets partial access if the data structure is being accessed in the preempted context, in which case the handler can access only the underlined fields to record its intended operation. Deferred operations run immediately before the code releases full access.

Data structure	Fields	Full access	Partial access	Deferred operation
Wait queue	Linked list `L` Counter `x`	1. Add a task to `L` 2. Remove the highest priority task from `L`	Increment `x`	Remove `x` high-priority tasks from `L` and clear `x`
Ready queue	Linked list `L` Lock-free buffer `I`	1. Add a task to `L` 2. Remove the highest priority task from `L`	Insert a task to `I`	Add tasks in `I` to `L` and clear `I`
Mailbox	Task placeholder `H` Counter `x`	1. Set `H` to hold a task or decrement `x` 2. Take the task from `H` or increment `x`	Increment `x`	Clear `H` and decrement `x` if `H` is not empty
Sleep queue	Linked list `L` Lock-free buffer `D`	1. Add a task to `L` 2. Remove expired tasks from `L`	Insert a task to `D`	Remove expired tasks and tasks in `D` from `L`, then clear `D`

Most of the implementation is architecture-independent. Only two small parts are not: the modification to the compiler to generate the function prologue and the inline assemblies in the library crate. Although all instructions supported by Cortex-M0 are also legal on M4, we use some M4-only instructions to reduce code size and improve performance on M4. Because M0 lacks atomic instructions, such as `ldrex` and `strex`, we implement atomic update operations for Cortex-M0 using global interrupt masking.

Compiler Modification. We modified both the rustc compiler frontend and the LLVM backend to support FS-semantics, and we have open-sourced the patches. To generate the function prologue (§4.1.1), we added the `split-stack` attribute to all functions during the intermediate representation (IR) emission stage in rustc. This attribute triggers a prologue adjustment step in LLVM's frame lowering stage, which we further modified to produce our desired prologue. To instrument drop handlers (§4.1.1), we modified the drop elaboration step in the mid-level IR (MIR) stage of rustc. In total, we added 260 lines of code to the frontend and 270 lines to the backend. We also removed 30 and 110 lines from the frontend and backend, respectively, to prevent optimizations based on the `nounwind` attribute (§4.1.1). Finally, we modified 50 lines in Rust's `core` library to strip away unused debug information fields in `PanicInfo`, which can reduce the compiled binary size by a few kilobytes.

Library Crate. We implemented Hopter as a Rust library crate consisting of approximately 12,200 lines of Rust code, of which fewer than 1,000 are unsafe Rust. Hopter resorts to unsafe Rust code only when certain functionalities cannot be expressed in safe Rust, such as stack extension and dynamic memory management. The unsafe code also includes inline assemblies for direct register access, bootstrap and interrupt handler entry points, and primitive memory operations such as `memset`. We also implemented two auxiliary library crates: one to provide Hopter with the `#[main]` and `#[handler]` attribute macros, and the other to allow applications to configure OS parameters. These consist of approximately 1,000 and 40 lines of safe Rust, respectively. In addition, Hopter depends on open-source Rust library crates for implementing spin locks [66], intrusive data structures [21], lock-free data structures [20], and accessing CPU core peripherals [33]. Although these crates contain unsafe code within their provided safe abstractions, they have been widely used and scrutinized by the Rust community with at least millions of downloads of each.

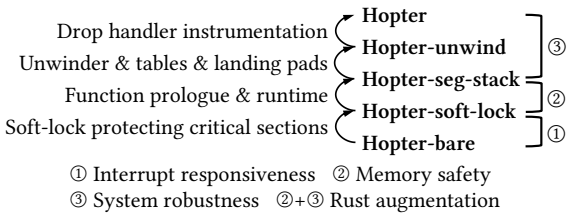


Figure 1: A breakdown of Hopter's unique features. The bottom "bare" version includes none of the components while the top includes all. Each version in the hierarchy has one more feature included compared to the one below it.

Feature Hierarchy. We made each Hopter's feature optional in a hierarchical manner, as shown in Figure 1. The hierarchy starts from the "bare" version that includes none of the features. Continuing up, the "soft-lock" version enables zero-latency interrupt handling by substituting soft-locks for interrupt masking in critical sections. The "seg-stack" version ensures stack memory safety and allows stack extension by generating prologues for each compiled function and incorporating a stack extension runtime. The "unwind" version supports resource reclamation upon Rust panic by incorporating the customized stack unwinder. The compiler also generates unwind tables and landing pads to assist the unwinder. Finally, the full Hopter version enables system resilience against stack overflows by instrumenting drop handlers and disabling compiler optimizations based on the `nounwind` attribute.

Atomic Operations for Cortex-M0. Because M0 does not support atomic instructions, particularly `ldrex` and `strex`, we implement atomic update methods on Cortex-M0 by globally disabling interrupts as Hopter needs these methods to implement soft-locks and to use the standard reference counting type `Arc`. These are compare-and-swap (CAS) and fetch-update (e.g., `fetch_add`), which are defined as methods on atomic types, such as `AtomicBool` and `AtomicPtr`, and atomic integer types, such as `AtomicU32`. Listing 4 shows the pseudo-code for the CAS logic. The code for the fetch-update operations is similar. Overall, this adds around 900 lines of code to the Rust `core` library.

```
fn compare_and_exchange(
    x: &mut AtomicU32, expect: u32, new: u32
) -> (bool, u32) {
    // Disable interrupts
    let irq_disabled = is_irq_disabled();
    disable_irq();

    // CAS logic
    let cur = x.val; let succ = false;
    if expect == cur { x.val = new; succ = true; }

    // Enable interrupts
    if !irq_disabled { enable_irq(); }
    return (succ, cur);
}
```

Listing 4: Pseudo-code for the atomic compare and exchange operation implemented for Cortex-M0. The operation compares the atomic integer's current value with the expected value, and if they match, store the new value. Disabling interrupts ensures the atomicity of the operation.

Implementing atomic update operations by disabling interrupts has a small impact on Hopter's interrupt handling latency, confirmed by our measurement results (Figure 2). More importantly, interrupts are disabled for only a short constant duration while executing the atomic logic, which is less than 10 instructions. Therefore, Hopter's interrupt handling latency is still constant bounded.

6 Evaluation

Hopter aims to provide system memory safety, robustness, and responsiveness, while requiring minimal application cooperation. Because memory safety is achieved by construction, this section quantifies robustness and responsiveness, as well as the overhead brought by the unique design choices of Hopter. Specifically, we seek to answer the following questions:

- (Q1 Overhead): What is the overhead introduced by Hopter in order to achieve its objectives?
- (Q2 Size): Is Hopter light enough to support microcontrollers with small flash and SRAM?
- (Q3 Responsiveness): Can Hopter support time-sensitive tasks and interrupt handling?
- (Q4 Robustness): Can Hopter recover from more fatal errors than other OSes and be fast enough for mission-critical applications?

6.1 Methods

We employ three orthogonal methods to answer these questions.

First, to quantify the impact by each of the features of Hopter, we follow the hierarchy in Figure 1 and measure the statistics for each listed variant.

Second, we compare Hopter with two well-known embedded OSes: FreeRTOS [1], the most widely used embedded OS, and Tock [41], the state-of-the-art Rust-based embedded OS. We develop a set of microbenchmarks with controlled workload for all three OSes. FreeRTOS employs no mechanisms to provide memory safety or robustness against application errors, and is designed to support high-frequency workloads and to be light-weight. Tock assumes application code can be malicious and employs hardware-based protection.

Finally, in order to assess Hopter's performance and overhead under a realistic application and demonstrate its ability to recover from errors under mission critical scenarios, we develop two embedded systems using Hopter, a flight control system for the Crazyflie [6] miniature drone and an Internet-of-Things (IoT) gateway.

6.2 Controlled Workload

We perform our micro-benchmarks with two evaluation boards: the STM32F412G-Discovery board equipped with an Arm Cortex-M4 CPU, 256 KiB SRAM, and 1 MiB flash, and the STM32F072B-Discovery board with an M0 CPU, 16 KiB SRAM, and 128 KiB flash. We configure the M4 CPU to run at 96 MHz and the M0 at 48 MHz, and we enable the floating point unit on M4. We set the compiler to optimize for speed (`-O3`). For our experiments, we first build an LED blinking application to show the minimum resource required when developing with an OS. Next, we quantify system responsiveness by measuring the latency of the task context switch and response to interrupts. Finally, we demonstrate that Hopter can recover from more complex scenarios than other OSes with a serial server task.

6.2.1 Minimum System (Q1/Q2). To measure the minimum hardware resources required to run a system developed using the three OSes under comparison, we build a blinking LED application, the "hello world" program for the embedded world. We assume an application programmer's role, i.e., using the OS and hardware abstraction layer (HAL) library as-is through their public APIs, without modifying their internal implementation. We use the HAL library from `stm32-rs` [30] and STM32Cube [61, 62] for Hopter and FreeRTOS, respectively. Tock comes with its own HAL.

The minimum system columns in Table 2 list the flash and SRAM size. The flash overhead of Hopter mainly comes from the stack unwinder logic, the unwind table, the landing pads, and the drop handler instrumentation. A minor overhead comes from the function prologue and stack extension runtime. The flash overhead can be amortized with a more sophisticated application like the flight control system (§6.3), for which we provide a more detailed breakdown of its 56% flash overhead.

The minimum hardware resources required to run Hopter without robustness features, i.e., the "bare", "soft-lock", or "seg-stack" variants, are similar to those of FreeRTOS. The support for stack unwinding and the drop handler instrumentation incur noticeable overhead particularly in flash usage, but the numbers are still within the lower tens of KiB. On the other hand, systems developed with Tock requires significantly larger flash and SRAM as shown in Table 2, because Tock is compiled separately from the application. The compiler toolchain is unable to remove unused OS code at compile time, resulting in the size bloat.

6.2.2 Task Scheduling and Interrupt Handling (Q1/Q3). To compare the system responsiveness, we measure the following performance metrics: the latency of a context switch between two tasks and the latency to respond to an interrupt from a handler or task.

Context switch latency reflects the overhead for inter-task cooperation. Hopter's latency is on the same order of magnitude as FreeRTOS's, allowing it to support multiple cooperating tasks

Table 2: Comparing Hopter to FreeRTOS and Tock, showing the overheads of Hopter's components, and showing the overhead of implementing atomic update operations by disabling interrupts. Hopter requires more flash than FreeRTOS mainly because it includes additional components to improve robustness. Hopter requires significantly less flash and memory than Tock that compiles the OS code separately. Hopter has the lowest and most consistent interrupt response latency, benefiting from the soft-lock. Hopter's context switch performance and task notification latency is close to FreeRTOS's and an order of magnitude lower than Tock's. Most flash and performance overhead of Hopter comes from the components for robustness, while the responsiveness and safety components introduce only small overhead. Hopter provides all three features on Cortex-M0 like on M4, but the implementation of atomic update operations by disabling interrupts increases the latency of context switch and interrupt handling by 20%.

		Minimum system		Latency (µs)			Feature		
		Flash (KiB)	†SRAM (KiB)	* Task	** Interrupt	** Interrupt-task	Responsive	Safe	Robust
OS	FreeRTOS	13.70	3.76	8.0	2.20 (0.39)	12.76 (0.73)	✔	✘	✘
	Tock	‡147.5 + 6.60	‡66.53 + 0.75	264.7	151.54 (26.03)	365.14 (26.71)	✘	✔	✔
	Hopter	**27.68**	**1.00**	**16.0**	**1.36 (0.01)**	**31.88 (2.44)**	✔	✔	✔
Component	Hopter-unwind	22.71	1.00	12.9	1.32 (0.03)	26.44 (1.96)	✔	✔	✘
	Hopter-seg-stack	12.60	0.84	13.0	1.20 (0.01)	27.24 (1.90)	✔	✔	✘
	Hopter-soft-lock	10.05	0.75	12.6	1.16 (0.02)	25.48 (1.84)	✔	✘	✘
	Hopter-bare	9.12	0.49	15.5	5.26 (1.21)	28.30 (1.83)	✘	✘	✘
Architecture	FreeRTOS (M0)	12.36	3.75	19.2	4.88 (1.15)	29.50 (4.43)	✔	✘	✘
	FreeRTOS (M0 on M4)	13.32	3.75	7.3	2.12 (0.43)	12.22 (1.66)	✔	✘	✘
	Hopter (M0)	25.60	0.92	48.7	3.18 (0.05)	76.20 (7.03)	✔	✔	✔
	Hopter (M0 on M4)	25.67	0.92	19.9	1.64 (0.02)	38.64 (2.83)	✔	✔	✔

†: Excluding function call stacks, whose sizes are configurable. ‡: OS plus application size.
*: Average from 10,000 trials. **: Maximum and standard deviation from 10,000 trials.
M0: Measured on STM32F072B-Discovery (Cortex-M0). Others are measured on STM32F412G-Discovery (Cortex-M4).
M0 on M4: Code is compiled with only M0 instructions but runs on STM32F412G-Discovery (M4).

with frequencies up to 1,000 Hz (§6.3). Tock on the other hand is substantially slower, because each context switch from a task to another requires four context switches between the OS and applications, rendering Tock not suitable for performance-demanding workloads. We compute context switch latency by running two tasks performing context switches to each other using the most efficient synchronization primitive available, i.e., mailbox on Hopter, task notification on FreeRTOS, and inter-process communication (IPC) on Tock. The task column in Table 2 lists the average latency by measuring 10,000 context switches using a hardware timer on the microcontroller precise to 1 µs. Hopter's latency is higher than FreeRTOS's mainly due to the use of safe Rust in implementing task lists and bookkeeping the current running task. The optimized organization and operations of task list found in FreeRTOS are incompatible with Rust's ownership model (see §2). However, an 8 µs overhead in context switch results in less than 1% CPU load increase for a task running at 1,000 Hz.

We also measure the interrupt response latency, which determines if the system may miss an ephemeral event. The measured board registers an interrupt handler to be triggered by a rising edge on a general-purpose input/output (GPIO) pin. The handler simply sets another GPIO pin to high to respond, either directly or by notifying a high-priority task to do so. We program another board to trigger the rising edge and measure the response latency with precision of 0.02 µs.

The Interrupt column in Table 2 reports the maximum latency observed in 10,000 tests in which the interrupt handler directly responds to the interrupt, while the Interrupt-task column reports that observed when the handler notifies a high-priority task to respond. Since Tock does not support declaring an application interrupt handler, we instead register the handler as a capsule in its kernel space. The latency numbers are obtained when two other tasks are context switching to each other as the system background workload. Hopter has the lowest and most consistent latency when responding in the interrupt handler, benefiting from the soft-lock mechanism. When handling the interrupt with a task, Hopter becomes slower than FreeRTOS due to its slower context switch. In both cases, Hopter is orders of magnitude faster than Tock.

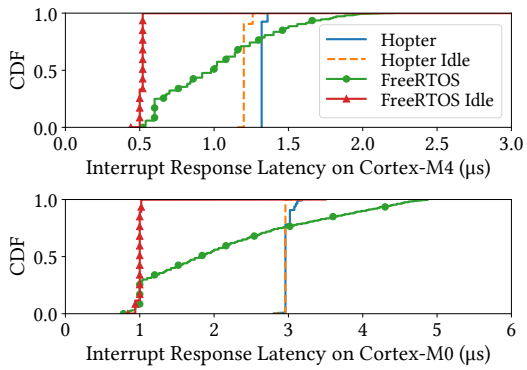


Figure 2: Hopter's interrupt response latency is almost constant using soft-lock on Cortex-M4. Background workload causes slight increase in latency due to the stress on instruction and literal cache. Hopter's latency on Cortex-M0 has small variance due to how it implements atomic operations on M0, namely disabling interrupts. In contrast, FreeRTOS's latency is sensitive to background workload, because it masks interrupts inside its critical sections.

Hopter's soft-lock is effective in maintaining a consistent interrupt response latency regardless of the background workload, as shown in Figure 2. The slight increase in the number under load is due to the eviction of cached instructions and literals in the ART

Figure 3: Crazyflie 2.1, a commercial off-the-shelf miniature drone. We implement a flight control system with Hopter to evaluate its capability of supporting real-time applications.

accelerator for flash [59]. In contrast, FreeRTOS's latency is sensitive to the background workload. Masking interrupts in its critical sections causes a delay in the interrupt response, and such a delay will be exacerbated if the CPU runs at a lower frequency as on M0 or the OS is under heavier load.

Hopter is responsive when running on M0, because the atomic update operations disable the interrupts for only a short constant period. The slowdown on M0 is mainly due to the CPU being less performant than M4. To measure the overhead caused by the longer function prologue and atomic update operations on M0, we run the same experiments on the M4 board but compile the code using only the instructions available on M0. Hopter's latency of context switch and interrupt response increases by 20%. FreeRTOS is slightly faster when running the M0 code on M4 than when running the M4 code because the M0 code does not preserve and restore floating-point registers during task context switch.

6.2.3 Fatal Error Recovery (Q4). Hopter allows a faulty application task to perform arbitrary cleanup logic during resource reclamation, which enables the system to recover from more complex scenarios and overcome the limitations of a simple task restart. We implement an application as a proof-of-concept consisting of three tasks: A serial server task that transmits data received from other tasks over the serial interface, and two client tasks that periodically send data to the server task. To prevent undesirable data interleaving between sending tasks, a client task first acquires a lock on the server before sending data and releases it after finishing.

We deliberately trigger an out-of-bounds array write in one client task while it is holding the lock. In Hopter, the fault manifests itself as a Rust panic, which causes the task to be restarted. During stack unwinding, the unwinder invokes the drop handler of the lock guard object to release the lock. After restart, both client tasks proceed as normal. In contrast, on Tock, if the written address falls in the task's allowed address range, it becomes a silent data corruption within the task. Otherwise, the Tock detects the fault and restarts the task, but the lock will not be released, subsequently causing a deadlock. Since FreeRTOS has no safety protection, the out-of-bounds write either causes a system-wide data corruption, or triggers a hardware fault that hangs or resets the system.

6.3 Drone Flight Control

We develop a flight control system with Hopter to measure its overhead in resource usage and CPU load with a realistic application, and to quantify the latency of interrupt response and task recovery demonstrating its suitability for real-time workload. The program runs with the commercially available Crazyflie 2.1 hardware platform [6] as shown in Figure 3. The drone is powered by

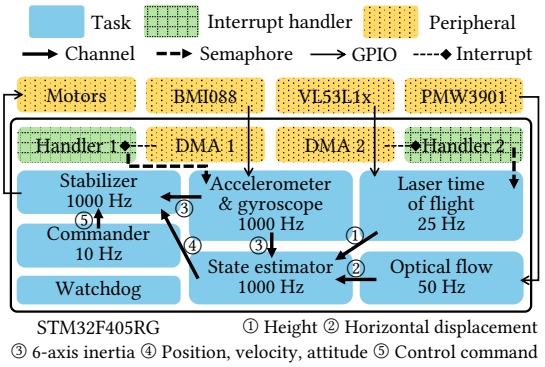


Figure 4: The flight control system consists of interrupt handlers and periodical tasks running at high frequency. Tasks and handlers synchronize through channels and semaphores. Peripherals are connected via GPIO pins.

Table 3: Binary size, SRAM usage, and CPU load of the flight control system. The SRAM usage excludes function call stacks, whose sizes are configurable. The CPU load is the average of 100 measurements when the drone is set to hover.

	Flash (KiB)	SRAM (KiB)	CPU
FreeRTOS	133.56	27.93	36.5%
Hopter	**213.53**	**7.51**	**52.3%**
Hopter-unwind	173.43	7.51	48.8%
Hopter-seg-stack	147.80	7.35	47.5%
Hopter-soft-lock	141.90	7.00	45.5%
Hopter-bare	136.95	6.52	48.4%

an STM32F405RG microcontroller, featuring a Cortex-M4F CPU with a maximum clock speed of 168 MHz, 192 KiB of SRAM, and 1 MiB of flash storage.

We implement the flight control system in Rust, which consists of around 10,000 lines of code. About 6,700 lines are peripheral driver code manually translated from open-source libraries [9, 51, 52]. We use the HAL library from `stm32-rs` [30] to access peripherals. The flight control system contains 22 unsafe Rust statements, mostly for interrupt configuration.

The flight control system includes seven periodical tasks that collectively manage flight control as shown in Figure 4. Three tasks are responsible for reading data from the inertial sensors, the height sensor, and the optical flow sensor, respectively. Two of them read through direct memory access (DMA). Subsequently, the State Estimator task obtains the collected data, through the data channel synchronization primitive provided by Hopter, and computes the drone's position and attitude with the Kalman filter algorithm. Finally, the Stabilizer task utilizes the output from the estimator to modulate the power distribution across the four propeller motors, aiming to maintain the drone's stability and follow the commands sent from the commander task. For operational safety, the Watchdog task supervises other flight control tasks, and if any of them is unresponsive for over a second, Watchdog will shut off all motors.

6.3.1 System Size and CPU Load (Q1/Q2/Q3). Table 3 shows the flash and SRAM usage of the flight control system and the CPU

load when the drone is set to hover. We break down Hopter's flash overhead as follows, where underlined numbers are constant, while others grow with the binary size. The function prologue for stack memory safety and the stack extension runtime each incur a flash overhead of 3.14 KiB and 2.75 KiB, respectively. To enable resource reclamation through unwinding, the stack unwinder adds 9.59 KiB, the landing pads add 7.50 KiB, and the unwind table adds 8.53 KiB. To allow recovery from stack overflows, instrumenting the drop handlers increases the code size by 27.80 KiB and the unwind table by 4.70 KiB. Disabling compiler optimizations based on the `nounwind` function attribute increases the code size and the unwind table size by another 2.97 KiB and 4.64 KiB, respectively.

We also compare against a FreeRTOS-based implementation with a similar program structure. FreeRTOS's version requires similar flash storage as with Hopter's "bare" variant. Its higher SRAM usage is due to the use of larger static buffers, and its lower CPU load is due to the availability of highly optimized math library and the DMA driver implementation for the SPI bus to access the optical flow sensor.

Hopter's drop handler instrumentation (§4.1.1) incurs the largest overhead in flash size and CPU load, which is counterintuitive since the instrumented code appears to be short and applies only to drop handler functions. However, we observe a cascading effect in the generated code that leads to size bloat and CPU load increase. (1) If the type of any field within a compound type implements `Drop`, the drop handler of both inner and outer types will be instrumented. (2) ARMv7, as a RISC architecture, must load the address of task local variables into registers before accessing them, resulting in longer instruction sequences. (3) There is more register spilling, further increasing the length of the instruction sequence. (4) The larger function body causes the compiler not to inline some drop handler functions, resulting in more function calls and returns. (5) An outlined function, in turn, includes additional prologue instructions.

However, drop handler instrumentation is indispensable for correct recovery from stack overflow despite the overhead. We perform a static stack depth analysis for the seven flight control tasks, ignoring indirect function calls through function pointers or trait objects (0.1% of all calls). Five tasks reach their respective maximum stack size inside a drop function. Without the instrumentation, initiating stack unwinding inside a drop handler will cause a system reset.

6.3.2 In-flight Interrupt Response Latency (Q3). We measure the interrupt response latency while setting the drone to hover, following the same experimental setup as in §6.2.2. With soft-lock, the maximum interrupt response latency over 10,000 measurements is 1.04 µs with a standard deviation of 0.02 µs. However, without soft-lock, the maximum latency rises to 7.56 µs with a standard deviation of 0.57 µs, due to the critical sections that prevent the system from responding to interrupts for an extensive period.

Prompt interrupt response is essential to support the radio on the drone. The drone employs an nRF51822 chip that forwards received radio packets to the main microcontroller via a universal asynchronous receiver-transmitter (UART) serial interface at baud rates of up to 2 Mbps. In the Syslink protocol [7] used by the drone, the length of the packet is determined by the first four bytes, so DMA can only be initiated after these bytes are received. Without DMA, the system must respond immediately to the interrupt after the UART interface receives a byte. Considering that each data byte carries an overhead of one start bit and one stop bit, an interrupt response delay over 5 µs will cause an overrun of the receive shift register [60], resulting in data loss. Only with soft-locks can the radio module function correctly.

Table 4: Recovery latency and stack unwinder workload for the Stabilizer task upon fatal errors. The error is either a deliberate `panic!()` or a stack overflow after some function recursions. Both the task context reuse and concurrent restart optimization can speed up the recovery upon panic and stack overflow. Concurrent restart maintains a constant recovery latency regardless of the stack depth.

	Panic	OF-recur-4	OF-recur-10
Unoptimized (µs)	197	268	402
Ctxt. reuse (µs)	134	207	335
Concur. restart (µs)	189	192	190
# Stack frames	6	7	13
# Landing pads	2	6	12

6.3.3 In-flight Fatal Error Tolerance (Q4). We deliberately introduce panics into the flight control system while the drone is hovering to assess Hopter's ability to recover the system from them. The drone has no visible disturbance when we introduce a panic into an interrupt handler or to the flight control tasks except the Stabilizer task. The drone rotates around 20 degrees along the yaw axis and drops a few centimeters when the Stabilizer task panics, but can still hover after the task restart. Stabilizer is the most sensitive to fatal errors because it directly modulates the power of the motors.

We also measure the recovery latency of Stabilizer on different fatal errors. For the panic test, we place a `panic!()` statement in the code that updates the stabilizer states, simulating out-of-bounds array access or a failed assertion. For the stack overflow test, we insert a call to a recursive function that defines a local object with a drop handler, which will overflow the stack after 4 or 10 recursions. Table 4 lists the latency, as well as the number of stack frames to unwind and landing pads to invoke for each test case. The recovery latency is measured as the time elapsed between the occurrence of the error and the execution of the entry function of the restarted task instance. A stack extension incurs an additional 20 µs delay.

The measurement result demonstrates that both the task context reuse and concurrent restart optimization can reduce recovery latency. Task context reuse outperforms concurrent restart when the stack is shallow, but concurrent restart enables faster recovery by maintaining a constant latency when the stack is deep.

In all three cases, Stabilizer can recover before the next execution interval. The observed disturbance to the drone is due to the new Stabilizer task instance not having any command to follow. The motors are kept at zero power until the next command is received.

6.4 IoT Gateway

We develop a gateway system for Internet-of-Things (IoT) devices using the STM32F072B-Discovery board to showcase Hopter's robustness against runtime errors, even on a Cortex-M0 microcontroller without an MPU. The gateway consists of a data plane and a control plane, as shown in Figure 5. The data plane receives packets from the inbound connection through an interrupt-driven UART

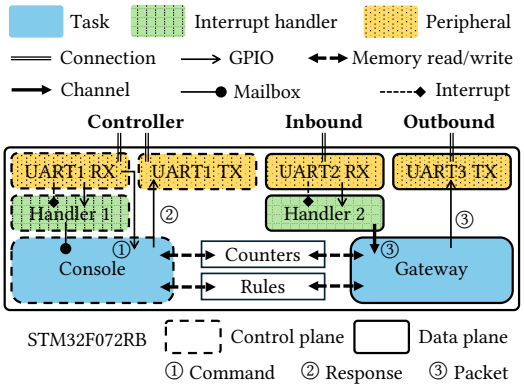


Figure 5: The gateway consists of a data plane and a control plane. The data plane forwards packets from the inbound connection to the outbound one. The control plane can gather runtime statistics and change forwarding rules. Errors in the control plane do not affect the operation of data plane thanks to Hopter's robustness features.

interface and forwards them to the outbound UART connection. As part of the data plane, the Gateway task verifies the integrity of received packets, updates runtime counters, and optionally modifies the packets according to the rules before forwarding them. The control plane establishes a text-based control session through another UART interface. Users can send commands to query the number of forwarded and corrupted packets, count occurrences of a specific byte pattern, reset the counters, and change forwarding rules. The rules allow users to specify escaped bytes and to filter packets based on their types. As part of the control plane, the Console task parses and executes the command, and responds to the user.

The gateway uses an open-source HAL library [29], consisting of 360 lines of Rust code, with only 8 lines unsafe for configuring peripherals. The compiled binary requires 60.0 KiB of flash and about 1.3 KiB of SRAM, excluding stacks. In comparison, an implementation based on FreeRTOS requires 21.0 KiB of flash and around 3.0 KiB of SRAM.

Hopter isolates data plane connections from fatal control plane errors. To trigger an error, we make the Console task skip checking the remaining free size of the buffer when receiving commands. A long command will overflow the buffer, causing the Console task to panic and restart. Meanwhile, the data plane continues to forward packets without disruption. The packet forwarding latency is unaffected by the panic recovery procedure. In contrast, the same buffer overflow error will silently corrupt data or crash a FreeRTOS-based system. Tock does not support this board due to the lack of an MPU.

7 Related Work

Memory Safety with Rust: Similar to Hopter's use of Rust, other OSes have leveraged Rust for memory safety across both the kernel and applications. Two recent works targeting PCs and servers, Theseus [8] and RedLeaf [50], utilize Rust to achieve memory safety, yet they still depend on the memory management unit (MMU) to place a guard page to prevent stack overflows. RIOT [5], Embassy [11], and RTIC [12] embedded OS, as well as the kernel space of Tock [41], employ Rust for memory safety on microcontrollers. RIOT supports threaded tasks written in Rust, but similarly uses MPU/PMP to set a guard region to prevent stack overflows. This approach will not allow recovery from stack overflows, as discussed in §4.1.2. Embassy, RTIC, and Tock's kernel space employs a single down-growing stack placed at the lower boundary of the SRAM region to prevent stack overflows from corrupting memory. However, they support only restricted scheduling patterns and cannot recover from stack overflows. In contrast, Hopter achieves memory safety solely via software, supports arbitrary scheduling patterns with threaded tasks, and can recover from stack overflows.

Fatal Error Recovery: Few embedded OSes for microcontrollers offer a recovery mechanism related to Hopter's capability of recovering from fatal errors. Tock [41] can automatically restart a task that failed due to memory access violations. Zephyr [27] can also terminate a task upon such an error but require the application to register an error handler separately. Neither supports automatic cleanup for graceful task termination. In contrast, Hopter runs application cleanup logic (drop handlers), allowing the system to avoid subsequent errors such as deadlocks.

Zero-latency Interrupt Handling: Hopter employs soft-locks for zero-latency interrupt handling, unlike previous embedded OSes such as eCos [47], PEASE [58], PURE [57], and SMX [48], which split interrupt handling into top and bottom halves to avoid OS interrupt masking. The split design necessitates a global serialization queue for deferred operations (bottom halves) that reference OS objects, therefore requiring unsafe Rust because the compiler cannot verify the lifetime of the references. Also, Hopter's soft-lock can improve performance by deferring operations only upon contention, which is rare, thus reducing CPU load by mostly taking the fast path.

8 Conclusion

This paper describes Hopter, a Rust-based embedded OS designed for microcontrollers. Hopter provides applications with memory safety, system robustness, and interrupt responsiveness while requiring minimal cooperation from them. Hopter forestalls stack overflows and converts such errors into Rust panics by executing Rust code under FS-semantics using compile-time code instrumentation and runtime OS support. By unifying all fatal errors as panics, Hopter performs stack unwinding to reclaim resources from failed application tasks or interrupt handlers, and can automatically restart failed tasks. To ensure timely response to interrupts, Hopter incorporates a novel mechanism called soft-locks to achieve zero-latency interrupt handling. Hopter is well-suited for resource-constrained microcontrollers and supports error recovery for real-time workloads. The mechanism for recovering from stack overflows is also applicable to processes running in a server, and soft-locks can be useful in reducing the latency to respond to process signals.

Acknowledgments

Zhiyao Ma, Guojun Chen and Lin Zhong were supported in part by National Science Foundation Award #2416594. Zhuo Chen was supported by Yale University. The authors were grateful for the useful feedback from their shepherd and reviewers.

References

[1] Amazon.com, Inc. 2020. FreeRTOS: Real-time operating system for microcontrollers and small microprocessors. https://www.freertos.org. Version 10.3.1. Accessed: 2025-04-24.

[2] Arm Limited. 2021. *ARMv7-M Architecture Reference Manual (Issue E.e).* https://developer.arm.com/documentation/ddi0403/ee/ Chapter B3.5: Protected Memory System Architecture, PMSAv7. Accessed: 2025-04-24.

[3] Arm Ltd. 2023. *Arm Compiler Version 6.6.5 armclang Reference Guide.* https://developer.arm.com/documentation/dui0774/l/ Section 2.28. Accessed: 2025-04-24.

[4] AWS open source. 2024. *FreeRTOS Documentation.* https://www.freertos.org/Documentation/00-Overview Chapter: FreeRTOS Stack Usage and Stack Overflow Checking. Accessed: 2025-04-24.

[5] Emmanuel Baccelli, Cenk Gündoğan, Oliver Hahm, Peter Kietzmann, Martine S Lenders, Hauke Petersen, Kaspar Schleiser, Thomas C Schmidt, and Matthias Wählisch. 2018. RIOT: An open source operating system for low-end embedded devices in the IoT. *IEEE Internet of Things Journal* (2018).

[6] Bitcraze. 2020. Crazyflie: A flying open development platform. https://www.bitcraze.io/.

[7] Bitcraze. 2024. *Syslink protocol version 2024.10.* https://www.bitcraze.io/documentation/repository/crazyflie2-nrf-firmware/2024.10/protocols/syslink/ Accessed: 2025-04-24.

[8] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. 2020. Theseus: an experiment in operating system structure and state management. In *Proc. USENIX OSDI.*

[9] Bosch. 2024. BMI088. https://github.com/BoschSensortec/BMI08x-Sensor-API. Version 1.9.0. Accessed: 2025-04-24.

[10] Ching-Han Chen, Ming-Yi Lin, and Chung-Chi Liu. 2018. Edge computing gateway of the industrial internet of things using multiple collaborative microcontrollers. *IEEE Network* (2018).

[11] Embassy Project Contributors. 2023. Embassy: The next-generation framework for embedded applications. https://embassy.dev. Accessed: 2025-04-24.

[12] RTIC Contributors. 2023. RTIC: The hardware accelerated Rust RTOS. https://rtic.rs/2/book/en/. Accessed: 2025-04-24.

[13] Jonathan Corbet. 2022. A first look at Rust in the 6.1 kernel. https://lwn.net/Articles/910762/. *Linux Weekly News* (10 2022).

[14] Microsoft Corporation. [n. d.]. Windows Drivers in Rust. https://github.com/microsoft/windows-drivers-rs. Accessed: 2025-04-24.

[15] MITRE Corporation. 2019. CVE-2019-25001: Flaw in CBOR deserializer allows stack overflow. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-25001. Accessed: 2025-04-24.

[16] MITRE Corporation. 2020. CVE-2020-35858: Parsing a specially crafted message can result in a stack overflow. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-35858. Accessed: 2025-04-24.

[17] MITRE Corporation. 2021. CVE-2021-31572: FreeRTOS before 10.4.3 has an integer overflow for a stream buffer. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-31572. Accessed: 2025-04-24.

[18] MITRE Corporation. 2021. CVE-2021-32020: FreeRTOS before 10.4.3 has insufficient bounds checking during management of heap memory. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-32020. Accessed: 2025-04-24.

[19] MITRE Corporation. 2024. CVE-2024-36760: A stack overflow vulnerability found in version 1.18.0 of rhai. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-36760. Accessed: 2025-04-24.

[20] Alex Crichton, Jeehoon Kang, Aaron Turon, and Taiki Endo. 2024. Tools for concurrent programming. https://crates.io/crates/crossbeam. Version 0.8.4. Accessed: 2025-04-24.

[21] Amanieu d'Antras. 2024. Intrusive collections for Rust (linked list and red-black tree). https://crates.io/crates/intrusive-collections. Version 0.9.7. Accessed: 2025-04-24.

[22] The Rust Security Advisory Database. 2018. RUSTSEC-2018-0005: Uncontrolled recursion leads to abort in deserialization. https://rustsec.org/advisories/RUSTSEC-2018-0005.html. Accessed: 2025-04-24.

[23] The Rust Security Advisory Database. 2023. RUSTSEC-2023-0080: Buffer overflow due to integer overflow in transpose. https://rustsec.org/advisories/RUSTSEC-2023-0080.html. Accessed: 2025-04-24.

[24] The Rust Security Advisory Database. 2024. RUSTSEC-2024-0012: Stack overflow during recursive JSON parsing. https://rustsec.org/advisories/RUSTSEC-2024-0012.html. Accessed: 2025-04-24.

[25] Hopter Developers. 2024. Hopter Embedded OS. https://github.com/hopter-project/hopter. Accessed: 2025-04-24.

[26] Redox Developers. 2015. Redox - Your Next(gen) OS. https://www.redox-os.org/. Accessed: 2025-04-24.

[27] Zephyr Developers. 2016. Zephyr Project. https://github.com/zephyrproject-rtos/zephyr. Accessed: 2025-04-24.

[28] Guo Dong, Wang Hongpei, Gao Song, and Wang Jing. 2011. Study on adaptive front-lighting system of automobile based on microcontroller. In *Proc. IEEE Int. Conf. Transportation, Mechanical, and Electrical Engineering (TMEE).*

[29] Daniel Egger. 2021. Peripheral access API for STM32F0 series microcontrollers. https://crates.io/crates/stm32f0xx-hal. Version 0.18.0. Accessed: 2025-04-24.

[30] Daniel Egger. 2024. Peripheral access API for STM32F4 series microcontrollers. https://crates.io/crates/stm32f4xx-hal. Version 0.22.1. Accessed: 2025-04-24.

[31] Bill Fleming. 2011. Microcontroller units in automobiles [automotive electronics]. *IEEE Vehicular Technology Magazine* (2011).

[32] Free Software Foundation, Inc. 2023. *GCC 15.0.0 Manual.* https://gcc.gnu.org/onlinedocs/gcc/ Section 3.12, Program Instrumentation Options. Accessed: 2025-04-24.

[33] Adam Greig. 2023. Low level access to Cortex-M processors. https://crates.io/crates/cortex-m. Version 0.7.7. Accessed: 2025-04-24.

[34] Mehedi Hasan, Maruf Hossain Anik, and Sharnali Islam. 2018. Microcontroller based smart home system with enhanced appliance switching capacity. In *Proc. IEEE HCT Information Technology Trends (ITT).*

[35] Robert Hieb, R Kent Dybvig, and Carl Bruggeman. 1990. Representing control in the presence of first-class continuations. *ACM SIGPLAN Notices* (1990).

[36] Free Software Foundation Inc. 2023. Exception handling and frame unwind runtime interface routines. https://github.com/gcc-mirror/gcc/blob/721cdcd1ddde0738deb08895e113a8db84187a14/libgcc/unwind.inc. Accessed: 2025-04-24.

[37] Fortune Business Insights. 2024. Embedded Systems Market Size, Share & COVID-19 Impact Analysis, By Component, By Type, By Application, and Regional Forecast, 2023-2030. https://www.fortunebusinessinsights.com/embedded-systems-market-108767. Accessed: 2025-04-24.

[38] Future Market Insights. 2024. Embedded System Market Analysis from 2024 to 2034. https://www.futuremarketinsights.com/reports/embedded-system-market. Accessed: 2025-04-24.

[39] Global Market Insights. 2024. Embedded Systems Market Size - By Component, By Application, By Function & Forecast, 2024 - 2032. https://www.gminsights.com/industry-analysis/embedded-system-market. Accessed: 2025-04-24.

[40] Prateek Khurana, Rajat Arora, and Manoj Kr Khurana. 2014. Microcontroller based implementation of Electronic Stability Control for automobiles. In *Proc. IEEE Int. Conf. Advances in Engineering & Technology Research.*

[41] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. 2017. Multiprogramming a 64kb computer safely and efficiently. In *Proc. ACM SOSP.*

[42] LLVM. 2025. AttributorAttributes.cpp - Attributes for Attributor deduction. https://llvm.org/doxygen/AttributorAttributes_8cpp_source.html. Accessed: 2025-04-24.

[43] LLVM. 2025. FunctionAttrs.cpp - Pass which marks functions attributes. https://llvm.org/doxygen/FunctionAttrs_8cpp_source.html. Accessed: 2025-04-24.

[44] Zhiyao Ma, Guojun Chen, and Lin Zhong. 2023. Panic Recovery in Rust-based Embedded Systems. In *Proc. ACM PLOS.*

[45] Zhiyao Ma and Lin Zhong. 2023. Bringing Segmented Stacks to Embedded Systems. In *Proc. ACM HotMobile.*

[46] Aleksander Malinowski and Hao Yu. 2011. Comparison of embedded system design for industrial applications. *IEEE Trans. Industrial Informatics* (2011).

[47] Anthony J Massa. 2002. *Embedded software development with eCos.* Prentice Hall Professional.

[48] Ralph Moore. 2005. Link Service Routines. *Micro Digital* (2005).

[49] Mohamed Abd El-Latif Mowad, Ahmed Fathy, Ahmed Hafez, et al. 2014. Smart home automated control system using android application and microcontroller. *Int. Journal of Scientific & Engineering Research* (2014).

[50] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. 2020. RedLeaf: Isolation and Communication in a Safe Operating System. In *Proc. USENIX OSDI.*

[51] Pimoroni. 2024. PMW3901. https://github.com/pimoroni/pmw3901-python/tree/main. Version 1.0.0. Accessed: 2025-04-24.

[52] Pololu. 2022. Vl53l1x. https://github.com/pololu/vl53l1x-arduino. Version 1.3.1. Accessed: 2025-04-24.

[53] LLVM Project. 2023. Implementation of C++ ABI Exception Handling Level 1. https://github.com/llvm/llvm-project/blob/f42482def236999b0f7896c09cd714b708861c8b/libunwind/src/UnwindLevel1.c. Accessed: 2025-04-24.

[54] RISC-V. 2024. *The RISC-V Instruction Set Manual: Volume II (Version 20240411).* https://riscv.org/technical/specifications/ Chapter 3.7: Physical Memory Protection. Accessed: 2025-04-24.

[55] Rust on Embedded Devices Working Group et al. [n. d.]. The Embedded Rust Book. https://docs.rust-embedded.org/book/. Accessed: 2025-04-24.

[56] Alice Ryhl. 2023. Rewriting Binder Driver in Rust. https://lore.kernel.org/rust-for-linux/20231101-rust-binder-v1-0-08ba9197f637@google.com/. Accessed: 2025-04-24.

[57] Friedrich Schon, Wolfgang Schroder-Preikschat, Olaf Spinczyk, and Ute Spinczyk. 2000. On interrupt-transparent synchronization in an embedded object-oriented operating system. In *Proc. IEEE Int. Symp. Object-Oriented Real-Time Distributed Computing (ISORC).*

[58] Wolfgang Schröder-Preikschat. 1994. *The logical design of parallel operating systems.* Prentice-Hall, Inc.

[59] STMicroelectronics. 2020. RM0402 (Rev.6) Reference manual for STM32F412 advanced Arm-based 32-bit MCUs. Chapter 3.4.2: Adaptive real-time memory accelerator (ART Accelerator).
[60] STMicroelectronics. 2024. *RM0090 (Rev.21) Reference manual for STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs.* https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf Chapter 30.3.3: Receiver - Overrun Error. Accessed: 2025-04-24.
[61] STMicroelectronics. 2024. STM32CubeF0 HAL driver MCU component. https://github.com/STMicroelectronics/STM32CubeF0/releases/tag/v1.11.5. Version 1.11.5. Accessed: 2025-04-24.
[62] STMicroelectronics. 2024. STM32CubeF4 HAL driver MCU component. https://github.com/STMicroelectronics/stm32f4xx_hal_driver/releases/tag/v1.8.3. Version 1.8.3. Accessed: 2025-04-24.
[63] Rust Tools Team. 2021. svd2rust. https://github.com/rust-embedded/svd2rust
[64] Theseus Developers. 2023. Support for unwinding the call stack and cleaning up stack frames. https://github.com/theseus-os/Theseus/blob/562a39cf6c662738f7718473f6bbc010970dce53/kernel/unwind/src/lib.rs. Accessed: 2025-04-24.
[65] Vishakha D Vaidya and Pinki Vishwakarma. 2018. A comparative analysis on smart home system to control, monitor and secure home, based on technologies like gsm, iot, bluetooth and pic microcontroller with zigbee modulation. In *Proc. IEEE Int. Conf. Smart City and Emerging Technology (ICSCET)*.
[66] Mathijs van de Nes and Joshua Barretto. 2023. Spin-based synchronization primitives. https://crates.io/crates/spin. Version 0.9.8. Accessed: 2025-04-24.
[67] Rob Von Behren, Jeremy Condit, Feng Zhou, George C Necula, and Eric Brewer. 2003. Capriccio: scalable threads for internet services. *ACM SIGOPS Operating Systems Review* (2003).

译文：

Hopter：一个安全、健壮且高响应能力的嵌入式操作系统

摘　要

基于微控制器的嵌入式系统容易受到内存安全问题影响，但是它们必须健壮且响应迅速，因为它们常被用于无人和关键任务场景。Rust编程语言通过编译时检查为内存安全提供了一个有吸引力的解决方案，但仍未能解决栈溢出问题，反而提升了中断处理延迟。我们提出了Hopter，一个基于Rust的嵌入式操作系统（OS），它在嵌入式系统中提供内存安全、系统健壮性和高中断响应能力，同时仅需最小程度的应用适配。Hopter通过一种新颖的有限堆栈语义，将栈溢出转换为Rust panic，从而能够通过栈展开和重启从致命错误中恢复。Hopter还采用了一种被称为软锁的新机制，使得操作系统从不禁用中断。我们使用受控工作负载将Hopter与其他知名嵌入式操作系统进行比较，并记录了我们使用Hopter开发微型无人机飞行控制系统和物联网（IoT）网关系统的经验。我们认为Hopter非常适合应用于资源受限的微控制器环境，并支持实时工作负载的错误恢复。

关键词：嵌入式系统，操作系统，Rust，内存安全，健壮性，中断响应能力

目　录

第 1 章 绪论

在过去的十年中，嵌入式系统的使用量日益增长，预计在未来几十年还将进一步增加更多[37-39]。由于资源限制，许多嵌入式系统采用没有内存管理单元（MMU）的微控制器，因此无法享受与虚拟内存相关的安全性。更糟糕的是，C语言作为一种内存不安全语言，一直是编程此类基于微控制器的嵌入式系统最常用的语言。因此，这些系统可能遭受与内存相关的安全漏洞，这些漏洞难以被发现，这一点从FreeRTOS[17, 18]发布十年后报告的漏洞中可见一斑。随着基于微控制器的嵌入式系统越来越多地被用于关键任务应用程序，在不要求更多资源或牺牲系统响应能力的前提下增强其内存安全性和系统健壮性势在必行。

Rust编程语言是一个有吸引力的C语言替代方案。它主要通过编译时检查来保证内存安全，仅增加少量运行时开销，并且已经在操作系统（OS）开发[8, 13, 14, 26, 50, 56]（包括嵌入式系统[41, 55, 63]）中得到采用。

然而，在基于微控制器的嵌入式系统中使用Rust也面临自身的挑战。首先，即使使用safe Rust，内存安全错误仍然存在，因为其语义假设函数调用栈具有无限大小，这对于没有虚拟内存且通常只有几十到几百KiB SRAM的基于微控制器的嵌入式系统来说尤其成问题。漏洞报告显示Rust库中存在栈溢出问题，即使是那些用于嵌入式用途的库[15, 16, 19, 22-24]。其次，Rust内置的异常处理机制，例如panic，需要一个栈展开器，而这在微控制器上通常不可用。没有栈展开器，panic会导致应用程序[41]或整个系统[11, 12]挂起或重置。最后，Rust的语言限制使得难以实现零延迟中断处理，即操作系统从不禁用中断以确保对事件的及时响应。已知的解决方案[47, 48, 57, 58]使用safe Rust是不可行的，因为它们难以通过编译时检查。§2详细阐述了这些挑战。

在本文中，我们试图回答以下问题：我们能否在为基于Rust的嵌入式系统带来内存安全、系统健壮性和快速中断的同时，仅要求应用程序最小程度的适配？ 我们通过Hopter嵌入式操作系统（§3）给出了肯定的答案。Hopter采用操作系统与实现语言的协同设计，以完善内存安全并实现错误恢复能力。Hopter还通过一种称为软锁的新型机制，为运行线程化任务的Rust语言编写的系统带来了零延迟中断处理能力。

Hopter通过有限堆栈语义（FS-语义）增强了Rust，实现了栈内存安全和溢出恢复能力（§4.1）。FS-语义将每次函数调用视为潜在的panic。在执行函数体之前，一

个prologue将检查剩余的栈空间，如果不足将触发panic。如果析构函数或其被调用者导致栈溢出，Hopter将临时扩展栈以完成析构函数[35, 45, 67]，然后触发panic。这有效地利用Rust panic机制将栈溢出与其他致命错误统一起来。

然后，Hopter利用自定义的栈展开器在panic时回收资源，这允许Hopter自动重启失败的任务（§4.2）。在可能的情况下，Hopter执行并发重启以加快恢复速度，其中新任务启动与失败任务的栈展开并发运行。

软锁通过对每个操作系统对象的修改进行序列化来实现零延迟中断处理，避免了全局序列化队列，从而避免了unsafe Rust代码（§4.3）。当任务或中断处理程序获取一个未被占用的软锁时，它将获得对受保护对象的完全访问权限。否则，它将获得部分访问权限以记录其计划在对象上执行的操作，软锁将在占用结束后提交这些操作。

我们将Hopter实现并开源为针对Arm Cortex-M0和Cortex-M4架构的Rust库（§5）。Hopter需要一个定制的编译器来编译系统，但Rust语法保持不变且语义兼容。Hopter还支持未经修改的第三方硬件抽象层（HAL）库，允许应用程序开发人员利用不断增长的Rust生态系统。我们通过与另外两个嵌入式操作系统，即FreeRTOS和Tock（§6），的比较来评估Hopter。Hopter具有比FreeRTOS更低的中断处理延迟，并且更安全、更健壮。Hopter比Tock需要更少的硬件资源，并且支持缺少内存保护单元（MPU）的微控制器。我们为Crazyflie 2.1微型无人机开发了一个飞行控制系统，并为物联网（IoT）设备开发了一个网关系统。使用飞行控制系统时，Hopter的闪存使用量增加了56%，FS-语义、栈展开机制和软锁共提升了15%的CPU占用率。网关系统展示了Hopter即使在无MPU的Cortex-M0 CPU上也能抵御运行时错误的健壮性。

我们做出以下贡献：

1. Rust的有限堆栈语义，通过编译时插桩和操作系统支持，保证栈内存安全和溢出恢复能力。

2. 软锁，一种新颖的同步原语，使得在基于Rust且运行线程化任务的系统上实现零延迟中断处理成为可能。

3. Hopter嵌入式操作系统的开源实现[25]，集成了FS-语义和软锁，以提供内存安全、故障恢复和快速响应，同时仅需应用最小程度的适配。

4. 对FS-语义、栈展开机制和软锁引起的可执行程序大小和性能开销的评估，表明Hopter适合应用于资源受限的微控制器和实时工作负载。

第 2 章 背景

微控制器广泛应用于嵌入式系统，从简单的家用电器[34, 49, 65]到复杂的汽车控制系统[28, 31, 40]和工业自动化系统[10, 46]。其中许多系统是无人值守或关键任务的，因此健壮性和从致命错误中恢复的能力至关重要。由于成本和能源限制，微控制器上可用的硬件资源通常有限。它们通常具有数百KB到几MB的闪存用于代码和只读数据，以及几十到几百KB的SRAM用于运行时数据。因此，为微控制器设计的操作系统必须是轻量级的。

微控制器在没有虚拟内存的情况下运行。所有代码都在单一的物理地址空间中运行，闪存、SRAM和外设寄存器都被映射到其中。CPU通常直接从字节寻址的闪存中执行指令，称为就地执行（XIP），而函数调用栈和可变数据则驻留在SRAM中。微控制器上的代码通常可以不受限制地访问整个地址空间，使得系统容易受到诸如缓冲区溢出和无效指针访问之类的内存安全错误的影响，这可能导致系统不稳定或引发安全漏洞。由于应用程序任务和操作系统之间缺乏隔离，攻击者可以利用内存安全漏洞轻松获得整个系统的完全控制。

基于硬件的内存保护，例如Arm上的内存保护单元（MPU）[2]和RISC-V上的物理内存保护（PMP）[54]，在高端微控制器上可用，并且已被一些嵌入式操作系统积极利用，例如Tock[41]。不幸的是，基于硬件的保护会带来代码大小、内存使用和运行性能方面的开销。MPU/PMP允许开发人员定义最多16个具有选择性读、写或执行权限的内存区域。然而，每个内存区域必须对齐并按最接近的2的幂大小调整，从而导致内部碎片而浪费闪存或SRAM。此外，使用MPU或PMP需要通过软件中断进行系统调用切换特权模式，参数需要经过封装并由操作系统验证，增加了性能开销。相比之下，在没有此类保护机制的系统中，系统调用可以有效地实现为简单的函数调用。因此，流行的嵌入式操作系统，如FreeRTOS[1]，即使是用C这样的不安全语言编写的，也将基于硬件的保护视为可选项。

Rust编程语言是一种现代系统编程语言，在提供对硬件的直接访问的同时，保证了内存和并发安全。它为基于硬件的保护提供了一个可观的替代方案，并且没有语言带来的显著开销。近年来Rust编程语言在操作系统开发[8, 13, 14, 26, 50, 56]和嵌入式系统[41, 55, 63]中的使用率有所增加。

Rust通过其所有权模型管理资源，而不使用垃圾收集器。每个值由一个变量拥有，当变量超出作用域时，它的析构函数被执行以释放资源。与其他语言通常在将值分配给新变量时进行复制或引用不同，Rust默认将值的所有权从旧变量移动到新变量。移动后，旧变量将不可访问。Rust中的函数调用也默认将参数值的所有权移动到被调用者，被调用者负责调用这些值的析构函数。因此，调用堆栈很可能在调用析构函数时被填满，这一现象已被我们的飞行控制系统（§6.3）所证实。

Rust强制使用所有权模型并在编译时分析引用的生命周期以确保内存安全，除非使用unsafe关键字。重要的是，编译时检查会拒绝一些常见的代码模式。例如，只有当引用在语法范围中的生命周期超过结构的生命周期时，Rust中的结构才能包含引用。此类限制阻碍了已知的零延迟中断处理实现[47, 48, 57, 58]，这种方法需要一个全局序列化队列来存储包含对各个操作系统对象引用的待处理操作。为了解决这个问题，Hopter引入了一种称为软锁的新颖机制（§4.3）来实现零延迟中断处理。

Rust通过一种称为panic的语言异常机制来补充其编译时检查，以阻止那些仅在运行时才能检测到的内存错误，例如数组越界访问。调用assert!或unwrap失败的断言也会导致panic。panic会终止Rust线程的正常执行流程。在个人电脑等硬件资源丰富的系统上，发生panic之后通常会进行栈展开，遍历调用堆栈中的函数帧并调用对象的析构函数以回收资源。这使得无需重启整个系统即可从错误中恢复。然而，嵌入式Rust系统通常缺少栈展开器[11, 12, 41]，因此panic将导致系统挂起或重置。更糟糕的是，断言在嵌入式Rust代码中很常见[30, 41]。为了系统健壮性，Hopter集成了一个为微控制器优化的栈展开器（§4.2）。

Rust内存安全的一个漏洞是函数调用堆栈仍然可能溢出。Rust语义假设每个运行的线程具有无限的栈空间，这在微控制器上尤其不现实。检测栈溢出的常见方法，包括栈保护者（金丝雀值）[3, 32]和定期栈指针检查[4]，都是事后补救措施。在检测到时，系统内存已经被破坏。先前的基于Rust的嵌入式系统要么使用MPU/PMP来防止栈溢出[41]，接纳相关的开销，要么仅使用单一的位于SRAM下边界的向下增长的堆栈[11, 12]，这限制了调度策略。Hopter通过在使用有限堆栈语义（§4.1）下运行Rust代码来解决这个问题，这不仅防止了栈溢出，还将此类错误转换为Rust panic，从而允许通过栈展开和重启进行统一的恢复过程。

第 3 章 使用 Hopter 进行系统开发

在介绍Hopter的设计（§4）和实现（§5）之前，我们先描述系统开发人员如何使用它开发嵌入式系统。

3.1 开发模型

Hopter是开源的，系统开发人员可以获取这个rust库。由于Hopter支持线程化任务并且放弃了基于硬件的保护，它可以与第三方HAL库[29, 30]进行互操作，显著降低了开发工作量。开发人员必须用Rust编写应用程序代码，并使用Hopter提供的定制Rust编译器来构建将Hopter作为依赖项的系统。下载并向cargo注册定制编译器仅需三个命令。

代码 3-1 应用程序代码从`[main]`属性标记的 main 函数开始执行。其他任务可以通过指定任务构建模式动态启动。当任务入口点闭包实现 Clone trait 时，该任务可以设置为可重启任务。

```
#[main]
fn main() {
    // 为另一任务初始化资源
    // 通过Arc封装使其实现Clone trait
    let res = Arc::new(init_resource());

    // 入口点闭包也拥有Clone trait
    // 可以设置为可重启任务
    task::build()
        .set_entry(move || another_task_main(res))
        .set_priority(8)
        .set_stack_limit(1024)
        .spawn_restartable()
        .unwrap();
}
```

开发人员像往常一样使用cargo build编译整个系统，Rust语法保持不变。

Hopter期望应用程序代码是无害的，因为应用程序代码可能使用unsafe Rust，因此引入内存错误。只要应用程序使用的所有unsafe代码都是正确的，Hopter就能保证系统的内存安全。这与流行的FreeRTOS[1]的期望类似；我们认为这是合理的，因为基于微控制器的嵌入式系统的系统开发人员通常可以访问并修改将要编译的所有源代码。Hopter的安全模型弱于Tock[41]，在Tock中应用程序代码可能是恶意的，操作系统必须使用基于硬件的保护（及其开销）来隔离自身（见§2）。

3.2 使用 Hopter 抽象

Hopter为系统开发人员提供了三个重要的抽象，以实现内存安全、容错能力和快速中断响应。为了实现应用逻辑，开发人员使用可重启任务或可容错中断处理程序上下文抽象，它们保证内存安全并允许从致命错误中恢复。可容错中断处理程序对硬件事件的响应具有零延迟，并且可以通过提供的同步原语与任务协作。这里我们详细阐述这三个关键抽象：其优点以及使用方式。它们的设计和实现细节将分别在§4和§5中介绍。

可重启任务：在Hopter上运行的任务是基于线程的，并且可被抢占式调度。它们通过自动重启提高了应用程序对致命错误的恢复能力。Hopter上的应用程序使用构建器模式（如代码3-2所示）生成任务，提供入口点闭包并指定任务属性。主任务是一个例外，它从标记有提供的#[main]属性的函数开始执行，该函数通常初始化系统并生成其他任务。要启用自动重启，应用程序任务只需确保其入口闭包实现了Clone trait，并使用spawn_restartable()方法启动。Clone trait允许Hopter复制重启任务所需的环境。若闭包中捕获的所有变量都实现了Clone trait，则该闭包就实现了Clone trait。当发生栈溢出、数组越界访问或断言失败等致命错误时，可重启任务会终止，释放其资源，并自动从头开始重新执行。可重启任务不会回滚到某个显式的检查点。取而代之的是，存储在堆中或用 static 声明的值会在任务重启后保持不变，而函数内的局部值则会被丢弃。如果入口点闭包不可克隆，仍可使用spawn()方法来运行任务，但该任务在发生致命错误时只会终止并释放资源，而不会重启。

代码 3-2 使用邮箱在中断处理程序和任务之间进行同步。中断处理程序通过#[handler(...)]宏声明。该处理程序具有零延迟响应时间，同时仍能够调用同步原语方法。第三方 HAL crate[30]提供外设访问。

```
// 引导后初始化为 'Some(_)。
// 自旋锁用于满足内部可变性规则。
static TIMER: Spin<Option<CounterUs<TIM2>>> = Spin::new(None);
static MAILBOX: Mailbox = Mailbox::new();

// 由定时器IRQ周期性调用。
#[handler(TIM2)]
fn tim2_handler() {
    TIMER.lock()
    .as_mut().unwrap() // 解包'Option'
    .wait().unwrap(); // 确认IRQ
    // 恢复任务执行。
    MAILBOX.notify_allow_isr();
}

// 作为任务生成。
fn another_task_main() {
    loop {
        MAILBOX.wait(); // 阻塞并让出CPU
        // 然后做其他事情 ...
    }
}
```

可容错中断处理程序：Hopter上的中断处理程序是响应硬件中断（IRQ）的函数，它们共享一个中断堆栈，并且总是抢占任务或更低优先级的中断处理程序。它们通过在中断处理期间容忍致命错误来提高系统健壮性。应用程序使用#[handler(IRQ_NAME)]属性宏（如代码3-2所示）声明中断处理程序函数。处理程序

在发生致命错误时终止并释放资源，如果中断仍然挂起，它将重新运行。由于中断上下文中动态内存分配的限制，Hopter无法从中断栈溢出中恢复。

同步原语：Hopter以Rust结构体类型的形式为应用程序提供同步原语，以方便上下文间协调。它们允许中断处理程序和任务之间进行同步，而不影响对中断的零延迟响应时间。应用程序代码通过调用定义的方法与同步原语交互。代码2展示了使用邮箱实现定时器中断处理程序与任务之间同步的示例。硬件定时器通过第三方HAL crate[30]访问。

第 4 章 系统设计

接下来，我们介绍Hopter实现内存安全、容错能力和中断响应性的关键设计方面。Hopter采用一种基于编译器的机制来保证可重启任务和可容错中断处理程序的内存安全。值得注意的是，对于栈内存安全，Hopter在编译时插桩代码（§4.1）的辅助下，使用有限堆栈语义（FS-语义）执行Rust代码。FS-语义将运行时内存错误和其他致命错误统一为Rust panic，允许Hopter应用基于栈展开和重启的通用恢复机制（§4.2）。此外，Hopter克服了与Rust语言相关的表达能力挑战，通过一种称为软锁的新颖机制实现零延迟中断处理（§4.3），以支持同步原语的实现。应用逻辑对编译时插桩和操作系统内部实现保持不可知，允许使用未经修改的现有Rust库。

4.1 具有有限堆栈语义的 Rust

Hopter使用有限堆栈语义（FS-语义）执行Rust代码，以确保栈内存安全并允许系统从栈溢出中恢复。FS-语义将每个Rust执行线程与一个隐式的堆栈大小相关联。Hopter通过panic（§4.1.1）来防止任何在没有足够空闲堆栈空间的情况下进行的函数调用。在析构函数执行期间发生溢出的特殊情况下，Hopter会临时扩展堆栈以完成析构函数，并在之后panic（§4.1.2）。Hopter采取两个步骤来实现FS-语义：

4.1.1 防止栈溢出

Hopter通过在执行函数体之前检查空闲堆栈空间来检测即将发生的栈溢出。这是通过编译器在分配堆栈帧的函数体之前插入的一段序言实现的。该序言计算空闲堆栈大小，即栈指针指示的当前栈顶与存储在任务局部变量BOUNDARY中的堆栈区域边界之间的差值。如果空闲空间不足，则陷入操作系统内核进行进一步诊断。函数序言的执行在Arm上最多需要8字节的保留堆栈空间。

Hopter在发生栈溢出的任务触发panic。该panic启动栈展开，从失败的任务中回收资源，以便在不泄露资源或死锁的情况下重启它。在通常情况下，Hopter在序言检测到即将发生的溢出时立即触发panic。陷入后，Hopter将任务的程序计数器设置为栈展开器入口以开始展开。

在析构函数执行期间发生栈溢出的罕见情况下，Hopter会扩展堆栈以在panic之前完成析构逻辑的运行。这是因为栈展开不能在析构函数内部开始，否则将跳过一些负责释放资源的代码。更准确地说，如果导致栈溢出的函数满足以下两个条件之

一，Hopter将扩展堆栈并延迟panic：（1）是一个析构函数（2）直接被或间接被一个析构函数调用。

这里需要两种独立的机制来检查这些标准。第一个标准由函数序言解决，它向Hopter传递触发溢出的函数是否是析构函数。

检查第二个标准需要对析构函数进行插桩。每当一个析构函数开始执行时，它将任务局部变量IN_DROP设置为true。析构函数在完成其逻辑后将该变量重置为false。在嵌套结构类型导致嵌套析构函数调用的场景中，只有最外层的析构函数在返回前清除该标志。Hopter可以通过检查IN_DROP标志的值来确定是否满足第二个标准。需要注意的是，函数序言仍然应用于析构函数插桩之上。

如果满足任一条件时发生溢出，Hopter会推迟panic。它将任务局部变量OVERFLOWED设置为true并扩展堆栈，而不是立即panic。在完成析构逻辑后，最外层的析构函数将检查OVERFLOWED标志，并在必要时触发被延迟的panic。

由于任何函数在FS-语义下都可能溢出堆栈并启动栈展开，这与一些编译器优化相冲突。LLVM编译器后端会推断函数的nounwind属性，并基于此简化生成的代码。如果函数满足以下两个条件[42, 43]，LLVM会将其标记为nounwind：（1）函数体不包含具有副作用的指令。（2）函数仅调用nounwind函数或者是叶子函数。然后，如果调用了一个nounwind函数，LLVM通过省略着陆板和展开表项来简化代码。然而，如果这样的函数调用导致栈溢出，栈展开将失败，导致挂起或系统重置。因此，Hopter为了正确性禁用了这些编译器优化。

4.1.2 扩展堆栈

Hopter利用分段堆栈[35, 45, 67]在没有虚拟内存的系统上扩展堆栈。分段堆栈是由称为堆栈块的非连续内存块组成的链表。如果剩余堆栈空间不足，它在函数调用时动态分配一个新的堆栈块，并在函数返回时释放它。

函数序言（§4.1.1）通过向Hopter提供请求的栈帧大小和传递参数大小来进行堆栈扩展。这两个数字是编译器在编译期间已知的常量，并作为字面量嵌入指令序列中，以避免使用额外的寄存器。Hopter随后通过跟踪陷入前的程序计数器来获取这些值，然后分配一个大小不小于这两个数字之和的堆栈块。如果存在传递参数，Hopter还会将它们复制到新分配的堆栈块中，以便它们与被调用者的栈帧相邻。最后，Hopter将任务的堆栈更改为新的堆栈块，并恢复任务，以便后者继续执行函数体。

我们注意到，堆栈扩展机制需要基于软件的溢出检测。这是因为堆栈的更改必须在执行函数体之前发生。相比之下，基于硬件的保护通常是在函数体开始执行并访问无效内存地址之后才检测到溢出。在函数执行中途将执行转移到新的堆栈块非常困难，因为复制栈帧会破坏内部引用。

4.2 从 panic 中恢复

Hopter支持从任务和中断处理程序中的Rust panic中恢复。panic可能由于数组越界访问、失败的断言或FS-语义下的栈溢出而被触发。

当任务或中断处理程序panic时，Hopter终止其执行并回收已分配的资源。panic被隔离在触发它的上下文中，不会阻止系统继续执行。如果任务的入口点闭包实现了Clone trait，Hopter可以在panic时自动重启它。因此，Hopter上的中断处理程序是可容错的，任务是可重启的。

4.2.1 展开堆栈

Hopter集成了一个栈展开器，用于在panic时回收资源，从而实现任务或中断处理程序的优雅终止，并允许后续恢复。展开过程在发生panic的任务或中断处理程序的上下文中运行，在此期间调度器可以继续执行上下文切换，更高优先级的中断可以在其上嵌套。从逻辑上讲，展开器在panic时强制活动函数立即返回，从调用堆栈的顶部开始，直到任务或中断处理程序的入口函数返回。在此过程中，局部对象被析构。从机制上讲，展开器引用编译器生成的展开表来恢复寄存器并调用称为着陆板的小段代码。着陆板有两种类型：调用析构函数的清理板和终止展开过程的捕获板。

Hopter的栈展开器与现有的展开器[36, 53, 64]在两个方面不同，以支持分段堆栈。首先，Hopter的展开器避免进行永不返回的分歧函数调用。由于展开器可能被调用来清理经历栈溢出的任务，它必须扩展堆栈以执行自身的逻辑。如果展开器进行分歧调用，所使用的堆栈块将不会被释放，从而导致内存泄漏。相比之下，先前的实现通常对着陆板进行分歧调用。其次，Hopter的展开器识别堆栈块边界，并在展开过程中释放堆栈块以防止内存泄漏。这很重要，因为任务的堆栈可能包含多个堆栈块，例如，当一个溢出的析构函数在扩展的堆栈块上运行引发延迟panic时。

4.2.2 重启应用程序

通过展开器回收资源后，Hopter可以通过从其入口点闭包重启来恢复发生panic的任务。为了启用重启，Hopter要求任务的入口点闭包实现Clone trait，以便安全地复制闭包所捕获的变量。对于中断处理程序，Hopter在捕获panic后执行异常返回，而不是重新执行处理程序。如果中断请求仍然挂起，中断处理程序将自动再次被调用。

为了加快从任务panic中的恢复，Hopter实现了[44]提出的并发任务重启优化。这种技术允许立即执行panic任务的新实例，与先前实例的展开过程并发运行，从而确保最小的无响应时间。Hopter将panic实例的任务优先级降至最低，允许展开过程利用空闲的CPU时间。Rust的所有权模型消除了重启实例与正在展开的实例之间的竞争条件。应用程序程序员可以选择退出并发重启，以防止重启的任务观察到逻辑上不一致的数据。在这种情况下，Hopter执行上下文重用优化，将已展开任务的任务结构重用于重启的实例。

4.3 零延迟中断处理

Hopter支持零延迟中断处理，在接收到中断时立即调用处理函数。这是可能的，因为Hopter从不禁用中断。为了防止中断处理程序与被抢占的上下文之间的竞争条件，Hopter采用了一种新颖的机制，称为软锁，它适合Rust的编译时检查。这些锁通过序列化来自并发上下文的修改来保护操作系统数据结构，允许Hopter中的中断处理程序与操作系统对象交互，例如，使用同步原语通知任务。

4.3.1 算法描述

软锁消除了因中断处理程序的并发访问而导致的竞态条件。获取软锁会产生对受保护数据结构的完全访问或部分访问。如果软锁尚未被获取，代码获得完全访问权限；否则获得部分访问权限。完全访问允许对所有数据结构字段进行修改，而部分访问仅允许修改部分字段。

当一个被抢占的上下文持有完全访问权限时，中断处理程序获得部分访问权限来记录其计划执行的操作。这些操作被推迟，直到处理程序返回且具有完全访问权限的代码完成其自身操作。为了进一步防止因并发任务执行而产生的竞态条件，调度器总是在获取软锁之前被挂起，因此在任务上下文中运行的代码总是获得完全访问权限。由于中断处理程序总是抢占任务并具有严格的优先级，具有完全访问权限

的代码在处理完具有部分访问权限的处理程序后恢复执行。因此，延迟操作可以在具有完全访问权限的代码完成其逻辑后立即无冲突地运行。

软锁封装了受保护的数据结构，并维护两个原子布尔标志来跟踪争用状态以及是否有任何延迟操作待处理。代码4-1展示了获取和释放软锁的伪代码。locked标志指示数据结构是否处于争用状态，pend标志指示是否有延迟操作待处理。受保护的数据结构必须实现AllowDeferredOp trait，以指定哪些字段可以通过完全或部分访问器访问，以及用于运行延迟操作的代码。在释放软锁时，会检查待处理的延迟操作，并在必要时使用作用域完全访问器执行它们，这使得它适合编译时检查。在实际实现中，软锁通过访问守卫的析构函数自动释放，而非手动释放。

4.3.2 Hopter 中的用例

软锁保护Hopter中的四种数据结构。表5-1列出了这些数据结构，以及完全或部分访问下的操作和延迟操作。等待队列为实现互斥锁、条件变量、信号量和通道等同步原语提供了基础。就绪队列为调度器维护就绪任务。邮箱是一种轻量级同步原语，允许任务等待通知，可以选择设置超时时间。休眠队列存储等待虚拟定时器到时的休眠任务。中断处理程序可以通过从等待队列或邮箱中移除任务、将其添加到就绪队列，并可选地从休眠队列中删除它来通知任务。

代码 4-1 软锁获取和释放操作的伪代码。受保护的数据结构必须实现 AllowDeferredOp trait，以指定哪些字段可以通过 Full 或 Partial 访问器访问，以及在运行 deferred_op()时执行什么代码。软锁在无争用时返回 Full 访问器，否则返回 Partial。延迟操作在释放 Full 访问器时执行。循环避免了错过在第一次检查 pend 标志之后到来的延迟操作。

```
struct SoftLock<T> where T: AllowDeferredOp{
    locked: AtomicBool, pend: AtomicBool, ...
}

fn acquire(sl: &SoftLock<T>) -> Access{
    let prev_locked = sl.locked.swap(true);
    if prev_locked { return Access::Partial; }
    else { return Access::Full; }
}
```

```
fn release(sl: &SoftLock<T>, acc: Access){
    match acc{
        Access::Partial => { sl.pend = true; }
        Access::Full => {
            loop{
                let prev_pend = sl.pend.swap(false);
                if prev_pend { acc. deferred_op(); }
                sl.locked = false;
                if !sl.pend { break; }
                else { sl.locked = true; }
            }
        }
    }
}
```

当一个受保护的数据结构处于争用状态时，处理程序使用部分访问来记录其计划执行的操作，这些操作被延迟到稍后执行。例如，考虑一个在邮箱上阻塞以等待硬件事件的任务。任务首先获取邮箱的完全访问权限以进入占位符H。如果中断发生在修改H期间，处理程序将获得对邮箱的部分访问权限，该权限仅允许其递增通知计数器x。处理程序返回后，任务恢复其操作，但在释放完全访问权限时会注意到递增的计数器。然后任务递减计数器并取消其阻塞。在另一种情况下，如果中断发生在任务完成其操作之后，任务将已经释放了完全访问权限并阻塞了自身。处理程序将获取邮箱的完全访问权限并在H中解除对任务的阻塞。

第 5 章 实现

我们已经为Arm Cortex-M4和Cortex-M0架构实现了Hopter，代码量约为15,000行。要在任一架构的微控制器系统上使用Hopter，开发人员只需定义一个中断向量数组，因为Hopter利用现有的HAL库进行外设访问。实现由两部分组成：编译器修改和Rust库代码。此外，我们以分层方式实现了Hopter的特性，以便开发人员可以在其收益和开销之间进行权衡。

表 5-1 由软锁保护的数据结构。如果软锁没有处于争用状态，代码获得对数据结构的完全访问权限以执行所列操作之一。在任务上下文中运行的代码总是获得完全访问权限。如果在被抢占的上下文中正在访问数据结构，中断处理程序获得部分访问权限，在这种情况下，处理程序只能访问带下划线的字段以记录其计划操作。延迟操作在代码释放完全访问权限之前立即运行。

数据结构	字段	完全访问	部分访问	延迟操作
等待队列	链表 L 计数器 x	1. 将任务添加到 L 2. 从 L 中移除最高优先级任务	递增 x	从 L 中移除 x 个最高优先级任务并清除 x
就绪队列	链表 L 无锁缓冲区 I	1. 将任务添加到 L 2. 从 L 中移除最高优先级任务	将任务插入 I	将 I 中的任务添加到 L 并清除 I
邮箱	任务占位符 H 计数器 x	1. 设置 H 以持有任务或递减 x 2. 从 H 中取出任务或递增 x	递增 x	如 H 不为空，则清除 H 并递减 x
休眠队列	链表 L 无锁缓冲区 D	1. 将任务添加到 L 2. 从 L 中移除到时任务	将任务插入 D	从 L 中移除到时任务和 D 中的任务，然后清除 D

大部分实现是架构无关的。只有两个很小的部分不是：为生成函数序言而对编译器的修改，以及库中的内联汇编。尽管Cortex-M0支持的所有指令在M4上也是合法的，但我们使用了一些仅M4的指令来减少代码大小并提高M4上的性能。由于M0缺少原子指令，如ldex和strex，我们使用全局中断屏蔽为Cortex-M0实现原子更新操作。

编译器修改：我们修改了rustc编译器前端和LLVM后端以支持FS-语义，我们开源了修改补丁。为了生成函数序言（§4.1.1），我们在rustc的中间表示（IR）生成阶段向所有函数添加了split-stack属性。该属性触发LLVM帧降低阶段中的序言调整，

我们进一步修改了该步骤以生成我们想要的序言。为了对析构函数进行插桩（§4.1.1），我们修改了rustc的中级IR（MIR）阶段的析构函数展开步骤。根据统计，我们向前端添加了260行代码，向后端添加了270行代码。我们还分别从前端和后端移除了30行和110行代码，以防止基于nounwind属性的优化（§4.1.1）。最后，我们修改了Rust核心库中的50行代码，以去除PanicInfo中未使用的调试信息字段，这可以将编译后的二进制文件大小减少几KB。

Rust库：我们将Hopter实现为一个Rust库，包含大约12,200行Rust代码，其中只有不到1,000行是unsafe Rust代码。Hopter仅在特定功能无法用safe Rust表达时才诉诸于unsafe Rust代码，例如堆栈扩展和动态内存管理。unsafe Rust代码还包括用于直接寄存器访问的内联汇编、引导、中断处理程序入口点以及原始内存操作（如memset）。我们还实现了两个辅助库：一个为Hopter提供#[main]和#[handler]属性宏，另一个允许应用程序配置操作系统参数。它们分别包含大约1,000行和40行的safe Rust代码。此外，Hopter依赖于开源的Rust库来实现自旋锁[66]、侵入式数据结构[21]、无锁数据结构[20]以及访问CPU核心外设[33]。尽管这些库在其提供的安全抽象内部包含unsafe Rust代码，但它们已被Rust社区广泛使用和审查，每个库的下载量至少达数百万次。

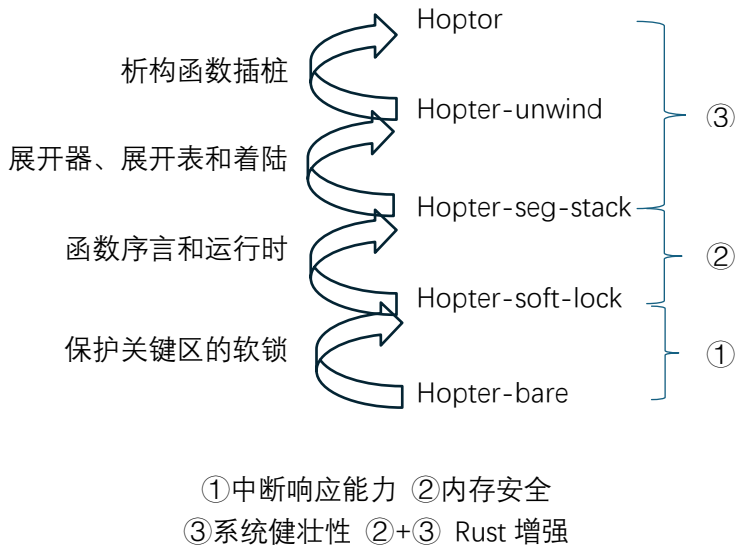


图 5-1 Hopter 特性的分解。底部的“基础”版本不包含任何组件，而顶部版本包含所有组件。层次结构中的每个版本比其下方的版本多包含一个特性。

特性层次结构：我们以分层方式使Hopter的每个特性成为可选的，如图5-1所示。该层次结构从"bare"版本开始，该版本不包含任何特性。向上，"soft-lock"版本通过用软锁替换临界区中的中断屏蔽来实现零延迟中断处理。"seg-stack"版本通过为每个编译函数生成序言并结合堆栈扩展运行时，确保栈内存安全并允许堆栈扩展。"unwind"版本通过结合定制的栈展开器，支持在Rust panic时回收资源。编译器还生成展开表和着陆板以协助展开器。最后，完整的Hopter版本通过对析构函数进行插桩并禁用基于nounwind属性的编译器优化，使得系统能够抵御栈溢出。

Cortex-M0的原子操作：由于M0不支持原子指令，特别是ldex和strex，我们通过在全局范围内禁用中断来实现Cortex-M0上的原子更新方法，因为Hopter需要这些方法来实现软锁和使用标准引用计数类型Arc。这些方法包含比较并交换（CAS）和获取并更新（例如，fetch_add），它们被定义为原子类型（如AtomicBool和AtomicPtr）和原子整数类型（如AtomicU32）上的方法。清单4展示了CAS逻辑的伪代码。获取并更新操作的代码类似。这向Rust core库添加了大约900行代码。

代码 5-1 为 Cortex-M0 实现的原子比较并交换操作的伪代码。该操作将原子整数的当前值与期望值进行比较，如果匹配，则存储新值。禁用中断确保了操作的原子性。

```
fn compare_and_exchange(
    x: &mut AtomicU32, expect: u32, new: u32
) -> (bool, u32) {
    // 禁用中断
    let irq_disabled = is_irq_disabled();
    disable_irq();

    // CAS 逻辑
    let cur = x.val; let succ = false;
    if expect == cur { x.val = new; succ = true; }

    // 启用中断
    if !irq_disabled { enable_irq(); }
    return (succ, cur);
```

```
}
```

通过禁用中断实现原子更新操作对Hopter的中断处理延迟影响很小，这已被我们的测量结果（图6-1）所证实。更重要的是，中断仅在执行原子逻辑的短时间内被禁用，这段时间少于10条指令。因此，Hopter的中断处理延迟仍然是常数量级的。

第 6 章 评估

Hopter旨在提供系统内存安全、健壮性和高响应能力，同时仅需应用最小程度的适配。由于内存安全是通过架构实现的，本节量化健壮性和响应能力，以及Hopter设计带来的开销。具体来说，我们试图回答以下问题：

（Q1 开销）：为实现其目标，Hopter引入了多少开销？

（Q2 大小）：Hopter是否足够轻量，以支持具有小闪存和SRAM的微控制器？

（Q3 响应能力）：Hopter能否支持时间敏感的任务和中断处理？

（Q4 健壮性）：Hopter能否比其他操作系统从更多致命错误中恢复，并且恢复的过程对于关键任务应用来说足够快？

6.1 方法

我们采用三种正交的方法来回答这些问题。

首先，为了量化Hopter每个特性的影响，我们遵循图1中的层次结构，并测量每个列出变体的统计数据。

其次，我们将Hopter与两个知名的嵌入式操作系统进行比较：最广泛使用的嵌入式操作系统FreeRTOS[1]，以及最先进的基于Rust的嵌入式操作系统Tock[41]。我们为所有三个操作系统开发了一套具有受控工作负载的微基准测试。FreeRTOS不采用任何机制来提供内存安全或抵御应用程序错误的健壮性，其设计旨在支持高频工作负载和轻量级。Tock假设应用程序代码可能是恶意的，并采用基于硬件的保护。

最后，为了评估Hopter在真实应用下的性能和开销，并展示其在关键任务场景下从错误中恢复的能力，我们使用Hopter开发了两个嵌入式系统：为Crazyflie[6]微型无人机开发的飞行控制系统和物联网（IoT）网关。

6.2 受控工作负载

我们使用两个评估板进行微基准测试：配备Arm Cortex-M4 CPU、256 KiB SRAM和1 MiB闪存的STM32F412G-Discovery板，以及配备M0 CPU、16 KiB SRAM和128 KiB闪存的STM32F072B-Discovery板。我们将M4 CPU配置为96 MHz工作频率，M0为48 MHz工作频率，并启用M4上的浮点单元。我们将编译器设置为优化运行速度（-O3）。在我们的实验中，我们首先构建一个LED闪烁应用程序，以显示使用操作

系统进行开发所需的最小资源。接下来，我们通过测量任务上下文切换的延迟和对中断的响应延迟来量化系统响应能力。最后，我们通过一个串行服务器任务证明Hopter能够从比其他操作系统更复杂的场景中恢复。

6.2.1 最小系统（Q1/Q2）

为了测量使用三种操作系统开发的系统所需的最小硬件资源，我们构建了一个闪烁LED应用程序，这是嵌入式世界的"Hello World"程序。我们扮演应用程序开发人员的角色，通过公共API使用操作系统和硬件抽象层（HAL）库，而不修改其内部实现。我们分别使用stm32-rs[30]和STM32Cube[61, 62]作为Hopter和FreeRTOS的HAL库。Tock有自己的HAL库。

表6-1中的"最小系统"一列列出了闪存和SRAM的大小。Hopter的闪存开销主要来自栈展开器逻辑、展开表、着陆板和析构函数插桩。函数序言和堆栈扩展运行时带来的开销较小。对于一个更复杂的应用程序（如飞行控制系统（§6.3）），闪存开销可以被平摊，我们提供了其56%闪存开销的详细分解。

运行没有健壮特性的Hopter（即"bare"、"soft-lock"或"seg-stack"版本）所需的最小硬件资源与FreeRTOS类似。对栈展开和析构函数插桩的支持产生了明显的开销，特别是在闪存使用方面，但仍然保持在几十KiB的范围内。另一方面，如表6-1所示，使用Tock开发的系统需要非常大的闪存和SRAM，因为Tock是与应用程序分开编译的。编译器工具链在编译时无法移除未使用的操作系统代码，导致体积膨胀。

6.2.2 任务调度和中断处理（Q1/Q3）

为了比较系统响应能力，我们测量以下性能指标：两个任务之间的上下文切换延迟以及处理程序或任务响应中断的延迟。

表 6-1 将 Hopter 与 FreeRTOS 和 Tock 进行比较，展示 Hopter 组件的开销，并展示通过禁用中断实现原子操作的开销。Hopter 需要比 FreeRTOS 更多的闪存，主要是因为它包含了额外的组件来提高健壮性。Hopter 比将操作系统代码分开编译的 Tock 需要显著更少的闪存和内存。得益于软锁，Hopter 具有最低且最稳定的中断响应延迟。Hopter 的上下文切换延迟和任务通知延迟接近 FreeRTOS，比 Tock 低一个数量级。Hopter 的大部分闪存和性能开销来自提供健壮性的组件，而响应能力和安全性组件仅引入很小的开销。Hopter 在 Cortex-M0 上像在 M4 上一样提供所有三个特性，但通过禁用中断实现原子操作，使上下文切换和中断处理的延迟增加了 20%。

	最小系统	延迟（μs）	特性

		闪存 (KiB)	†SRAM (KiB)	*任务	**中断	**中断-任务	高响应能力	安全	健壮
操作系统	FreeRTOS	13.70	3.76	8.0	2.20(0.39)	12.76(0.73)	✓	×	×
	Tock	‡147.5+6.60	‡66.53+0.75	264.7	151.54(26.03)	365.14(26.71)	×	✓	✓
	Hopter	**27.68**	**1.00**	**16.0**	**1.36(0.01)**	**31.88(2.44)**	✓	✓	✓
组件	Hopter-unwind	22.71	1.00	12.9	1.32(0.03)	26.44(1.96)	✓	✓	×
	Hopter-seg-stack	12.60	0.84	13.0	1.20(0.01)	27.24(1.90)	✓	✓	×
	Hopter-soft-lock	10.05	0.75	12.6	1.16(0.02)	25.48(1.84)	✓	×	×
	Hopter-bare	9.12	0.49	15.5	5.26(1.21)	28.30(1.83)	×	×	×
架构	FreeRTOS (M0)	12.36	3.75	19.2	4.88 (1.15)	29.50 (4.43)	✓	×	×
	FreeRTOS (M0 on M4)	13.32	3.75	7.3	2.12 (0.43)	12.22 (1.66)	✓	×	×
	Hopter (M0)	25.60	0.92	48.7	3.18 (0.05)	76.20 (7.03)	✓	✓	✓
	Hopter (M0 on M4)	25.67	0.92	19.9	1.64 (0.02)	38.64 (2.83)	✓	✓	✓

†：不包括函数调用堆栈，其大小可配置。‡：操作系统加上应用程序大小。

*：10,000次试验的平均值。**：10,000次试验的最大值和标准差。

M0：在STM32F072B-Discovery (Cortex-M0)上测量。其他在STM32F412G-Discovery (Cortex-M4)上测量。M0 on M4：代码仅使用M0指令编译，但在STM32F412G-Discovery (M4)上运行。

上下文切换延迟反映了任务间协作的开销。Hopter的延迟与FreeRTOS处于同一数量级，使其能够在高达1000Hz的频率下支持多个协作任务。而Tock显著更慢，因为每次任务上下文的切换需要在操作系统和应用程序之间进行四次上下文切换，导致Tock不适合用于性能要求高的工作负载。我们计算上下文切换延迟的方法是让两个任务使用最有效的可用同步原语（即Hopter上的邮箱、FreeRTOS上的任务通知、Tock上的进程间通信（IPC））相互进行上下文切换。表6-1中的"任务"一列通过使用微控制器上的精确到1 μs的硬件计时器测量10,000次上下文切换，得到了平均延迟。Hopter的延迟高于FreeRTOS，主要是由于在实现任务列表和记录当前运行任务时使用了safe Rust。在FreeRTOS中找到的任务列表的组织和操作优化与Rust的所有权模型不兼容（见§2）。然而，对于以1,000 Hz运行的任务，上下文切换中8 μs的开销导致CPU负载增加不到1%。

我们还测量了中断响应延迟，它决定了系统是否会错过短暂事件。被测板注册了一个中断处理程序，由通用输入/输出（GPIO）引脚的上升沿触发。处理程序只是将另一个GPIO引脚设置为高电平来响应，可以直接设置，也可以通过通知高优先级任务来设置。我们编程另一块板来触发上升沿，并以0.02 μs的精度测量响应延迟。

表6-1中的"中断"一列展示了在处理程序直接响应中断的10,000次测试中观察到的最大延迟，而"中断-任务"一列展示了在处理程序通知高优先级任务时观察到的最大延迟。由于Tock不支持定义应用程序中断处理程序，我们改为将处理程序注册为其内核空间中的一个capsule。当我们通过两个任务作为系统后台工作负载相互上下文切换测量获得延迟。得益于软锁机制，Hopter在中断处理程序中响应时具有最低且最稳定的延迟。当用任务处理中断时，由于上下文切换较慢，Hopter变得比FreeRTOS慢。在这两种情况下，Hopter都比Tock快几个数量级。

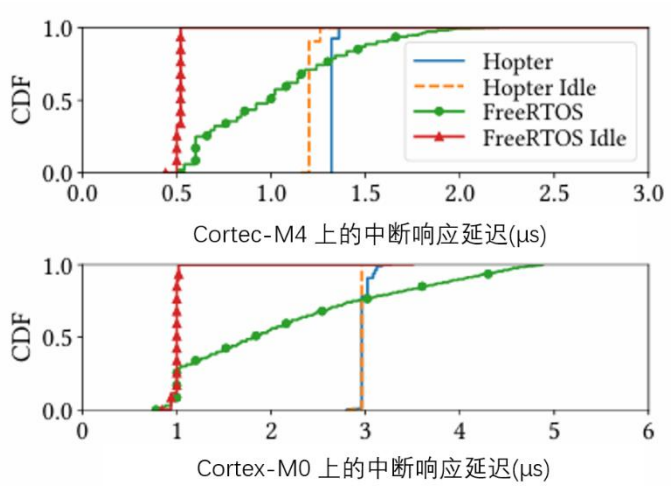


图 6-1 在 Cortex-M4 上使用软锁时，Hopter 的中断响应延迟几乎是恒定的。后台工作负载由于对指令和字面值缓存的压力导致延迟略有增加。由于在 M0 上实现原子操作（即禁用中断）的方式，Hopter 在 Cortex-M0 上的延迟有较小的方差。相比之下，FreeRTOS 的延迟对后台工作负载敏感，因为它在关键部分内屏蔽了中断。

Hopter的软锁在保持稳定的中断响应延迟方面是有效的，无论后台工作负载如何，如图6-1所示。负载下数只的略微增加是由于闪存中ART加速器[59]中缓存的指令和字面值的替换。相比之下，FreeRTOS的延迟对后台工作负载敏感。在其关键部分屏蔽中断会导致中断响应出现延迟，如果CPU以较低的频率（如在M0上）运行或操作系统负载较重，这种延迟会加剧。

Hopter在M0上运行时响应迅速，因为原子操作仅在短暂的常数时间内禁用中断。M0上的速度下降主要是由于CPU性能不如M4。为了测量由更长的函数序言和M0上的原子操作引起的开销，我们在M4板上运行相同的实验，但仅使用M0上可用的指令编译代码。Hopter的上下文切换和中断响应延迟增加了20%。当在M4上运行M0代码时，FreeRTOS比运行M4代码时稍快，因为M0代码在任务上下文切换期间不保存和恢复浮点寄存器。

6.2.3 致命错误恢复（Q4）

Hopter允许有故障的应用程序任务在资源回收期间执行任意的清理逻辑，这使得系统能够从更复杂的场景中恢复，并克服简单任务重启的限制。我们实现了一个概念验证应用程序，它由三个任务组成：一个通过串行接口从其他任务接收数据的串行服务器任务，以及两个定期向服务器任务发送数据的客户端任务。为了防止发送任务之间的不良数据交错，客户端任务在发送数据前首先获取服务器的锁，并在完成后释放它。

我们故意在一个客户端任务持有锁时触发一个越界数组写入。在Hopter中，该故障表现为Rust panic，导致任务被重启。在任务展开期间，栈展开器调用锁守卫对象的析构函数来释放锁。重启后，两个客户端任务都正常进行。相比之下，在Tock上，如果写入地址落在任务允许的地址范围内，它将成为任务内部的静默数据损坏。否则，Tock检测到故障并重启任务，但锁不会被释放，随后导致死锁。由于FreeRTOS没有安全保护，越界写入要么导致系统范围内的数据损坏，要么触发导致系统挂起或重置的硬件故障。

6.3 无人机飞行控制

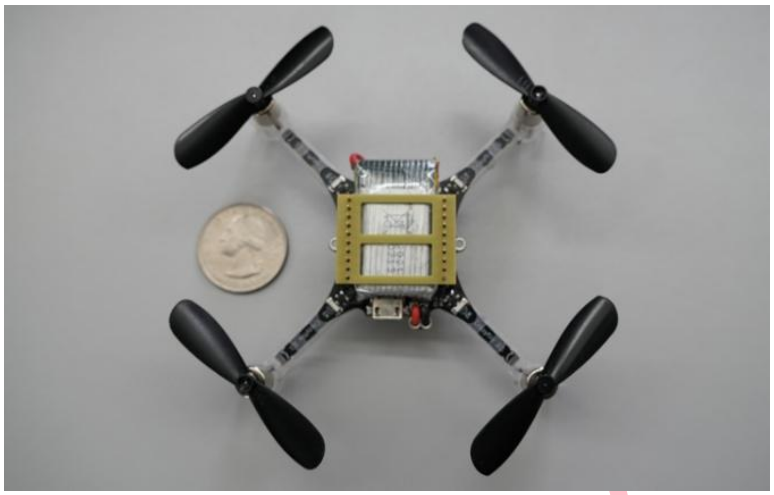


图6-2 Crazyflie 2.1，一款商业现成的微型无人机。我们实现了一个使用 Hopter 的飞行控制系统，以评估其支持实时应用的能力。

我们使用Hopter开发了一个飞行控制系统，以在真实应用中测量其资源使用和CPU负载的开销，并量化中断响应和任务恢复的延迟，展示其对实时工作负载的适用性。该程序在商业可用的Crazyflie 2.1硬件平台[6]上运行，如图6-2所示。无人机由STM32F405RG微控制器驱动，该微控制器具有最大时钟速度为168 MHz的Cortex-M4F CPU、192 KiB的SRAM和1 MiB的闪存存储。

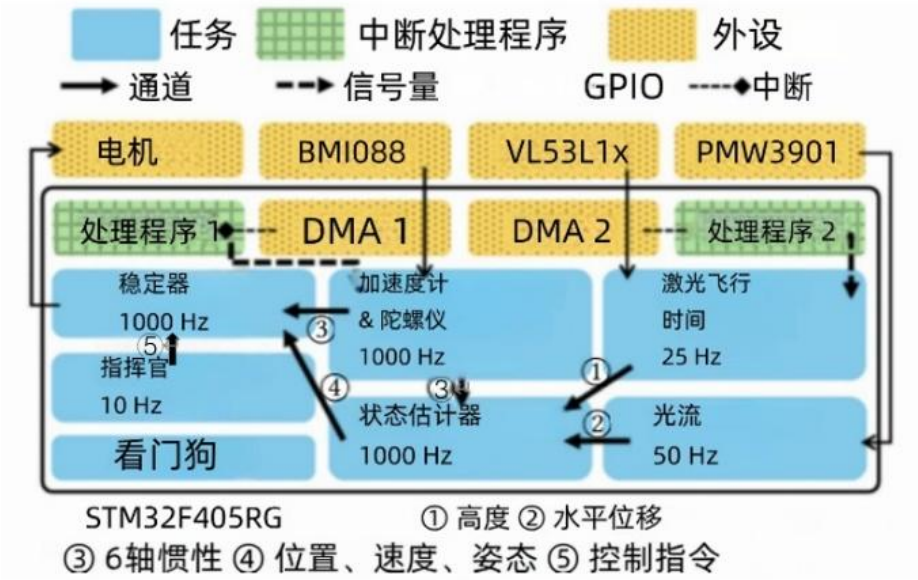


图 6-3 飞行控制系统由中断处理程序和高频率运行的周期性任务组成。任务和处理程序通过通道和信号量同步。外设通过 GPIO 引脚连接。

表 6-2 飞行控制系统的二进制大小、SRAM 使用和 CPU 负载。SRAM 使用不包括大小可配置的函数调用堆栈。CPU 负载是无人机悬停时 100 次测量的平均值。

	闪存 (KiB)	SRAM (KiB)	CPU
FreeRTOS	133.56	27.93	36.5%
Hopter	213.53	7.51	52.3%
Hopter-unwind	173.43	7.51	48.8%
Hopter-seg-stack	147.80	7.35	47.5%
Hopter-soft-lock	141.90	7.00	45.5%
Hopter-bare	136.95	6.52	48.4%

我们用Rust实现了飞行控制系统，大约包含10,000行代码。其中约6,700行是从开源库[9, 51, 52]手动翻译过来的外设驱动程序代码。我们使用stm32-rs[30]的HAL库来访问外设。飞行控制系统包含22条不安全的Rust语句，主要用于中断配置。

如图6-3所示，飞行控制系统包括七个周期性任务，它们共同管理飞行控制。三个任务分别负责从惯性传感器、高度传感器和光流传感器读取数据。其中两个通过直接内存访问（DMA）读取。随后，状态评估器任务通过Hopter提供的数据通道同步原语获取收集的数据，并使用卡尔曼滤波算法计算无人机的位置和姿态。最后，稳定器任务利用评估器的输出来调节四个螺旋桨电机的功率分配，旨在保持无人机的稳定性并遵循来自指挥者任务的命令。为了操作安全，看门狗任务监控其他飞行控制任务，如果任何任务超过一秒钟无响应，看门狗将关闭所有电机。

6.3.1 系统大小和 CPU 负载（Q1/Q2/Q3）

表6-2显示了飞行控制系统的闪存和SRAM使用情况，以及无人机悬停时的CPU负载。我们对Hopter的闪存开销进行了如下分解，其中带下划线的数字是常数，而其他数字随二进制文件大小增长。用于栈内存安全的函数序言和堆栈扩展运行时分别带来3.14 KiB和2.75 KiB的闪存开销。为了能够通过栈展开实现资源回收，栈展开器增加了9.59 KiB，着陆板增加了7.50 KiB，展开表增加了8.53 KiB。为了允许从栈溢出中恢复，析构函数插桩使代码大小增加了27.80 KiB，展开表大小增加了4.70 KiB。禁用基于nounwind函数属性的编译器优化使代码大小和展开表大小分别再增加了2.97 KiB和4.64 KiB。

我们还与一个具有类似程序结构的基于FreeRTOS的实现进行了比较。FreeRTOS的版本需要与Hopter的"bare"版本类似的闪存存储。其较高的SRAM使用是由于使用

了较大的静态缓冲区，其较低的CPU负载是由于使用了高度优化的数学库和用于访问光流传感器的SPI总线的DMA驱动程序实现。

Hopter的析构函数插桩（§4.1.1）在闪存大小和CPU负载方面产生了最大的开销，这是违反直觉的，因为插桩的代码似乎很短并且只应用于析构函数。然而，我们在生成的代码中观察到一个级联效应，导致体积膨胀和CPU负载增加。(1) 如果组合类型中任何字段的类型实现了drop，则内部和外部类型的析构函数都会被插桩。(2) ARMv7作为一种RISC架构，在访问任务局部变量之前必须将其地址加载到寄存器中，导致指令序列更长。(3) 有更多的寄存器溢出，进一步增加了指令序列的长度。(4) 更大的函数体导致编译器不选择内联一些析构函数，导致更多的函数调用和返回。(5) 一个外联的函数又包含了额外的序言。

然而，尽管有开销，析构函数插桩对于从栈溢出的正确恢复是必不可少的。我们对七个飞行控制任务进行了静态栈深度分析，忽略通过函数指针或trait对象的间接函数调用（占所有调用的0.1%）。五个任务在析构函数内部达到各自的最大栈大小。如果没有插桩，在析构函数内部启动栈展开将导致系统重置。

6.3.2 飞行中中断响应延迟（Q3）

我们按照与§6.2.2相同的实验设置，在设置无人机悬停时测量中断响应延迟。使用软锁，10,000次测量中的最大中断响应延迟为1.04 μs，标准差为0.02 μs。然而，如果没有软锁，由于存在禁用中断响应的临界区，最大延迟上升到7.56 μs，标准差为0.57 μs。

及时的中断响应对于无人机上的无线电至关重要。无人机采用一个nRF51822芯片，通过一个通用异步收发器（UART）串行接口（波特率高达2 Mbps）将接收到的无线电数据包转发到主微控制器。在无人机使用的Syslink协议[7]中，数据包的长度由前四个字节决定，因此只有在接收到这些字节后才能启动DMA。没有DMA，系统必须在UART接口接收到一个字节后立即响应中断。

表 6-3 稳定器任务在致命错误时的恢复延迟和栈展开器工作量。错误是故意使用 panic!()或某些函数递归后的栈溢出导致的。任务上下文重用和并发重启优化都可以加快在 panic 和栈溢出时的恢复。无论栈深度如何，并发重启都能保持恒定的恢复延迟。

	Panic	OF-recur-4	OF-recur-10
未优化（μs）	197	268	402
上下文重用（μs）	134	207	335

并发重启（μs）	189	192	190
#栈帧	6	7	13
#着陆板	2	6	12

考虑到每个数据字节携带一个起始位和一个停止位的开销，超过5 μs的中断响应延迟将导致接收移位寄存器[60]超限，从而导致数据丢失。只有使用软锁，无线电模块才能正常工作。

6.3.3 飞行中致命错误容错（Q4）

我们故意在无人机悬停时向飞行控制系统引入panic，以评估Hopter从致命错误中恢复系统的能力。当我们将panic引入中断处理程序或除稳定器任务外的飞行控制任务时，无人机没有可见的扰动。当稳定器任务发生panic时，无人机会绕偏航轴旋转约20度并下降几厘米，但在任务重启后仍能悬停。稳定器对致命错误最敏感，因为它直接调节电机的功率。

我们还测量了稳定器在不同致命错误下的恢复延迟。对于panic测试，我们在更新稳定器状态的代码中放置一个panic!()语句，模拟数组越界访问或失败的断言。对于栈溢出测试，我们插入一个对递归函数的调用，该函数定义一个具有析构函数的局部对象，该对象在4次或10次递归后将导致栈溢出。表6-3列出了每种测试用例的延迟，以及需要展开的栈帧数量和调用的着陆板数量。恢复延迟是从错误发生到重启任务实例的入口函数执行之间经过的时间。堆栈扩展会产生额外的20 μs延迟。

测量结果表明，任务上下文重用和并发重启优化都能减少恢复延迟。当栈较浅时，任务上下文重用优于并发重启，但当栈较深时，并发重启通过保持恒定的延迟实现了更快的恢复。

在所有三种情况下，稳定器都能在下一次执行间隔之前恢复。观察到的无人机的扰动是由于新的稳定器任务实例没有任何命令可以执行。电机保持在零功率，直到接收到下一个命令。

6.4 IoT 网关

我们使用STM32F072B-Discovery板为物联网（IoT）设备开发了一个网关系统，以展示Hopter即使在无MPU的Cortex-M0微控制器上也能抵御运行时错误的健壮性。该网关由一个数据平面和一个控制平面组成，如图6-4所示。数据平面通过一个中断

驱动的UART接口接收来自入站连接的数据包，并将它们转发到出站的UART连接。作为数据平面的一部分，网关任务验证接收到的数据包的完整性，更新运行时计数器，并可能在转发前根据规则修改数据包。控制平面通过另一个UART接口建立基于文本的控制会话。用户可以发送命令来查询已转发和损坏的数据包数量、统计特定字节模式的出现次数、重置计数器以及更改转发规则。这些规则允许用户指定转义字节，并根据数据包类型过滤数据包。作为控制平面的一部分，控制台任务解析和执行命令，并响应用户操作。

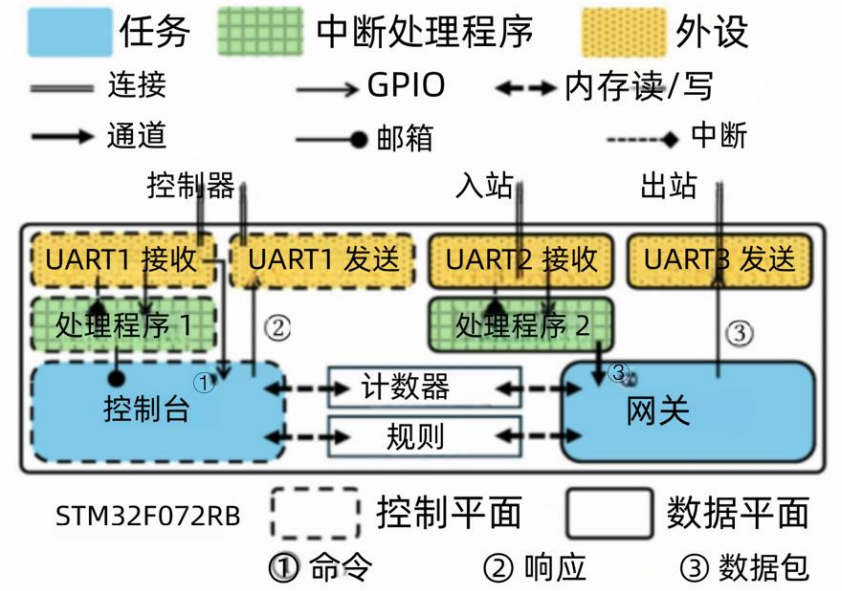


图 6-4 该网关由数据平面和控制平面组成。数据平面将数据包从入站连接转发到出站连接。得益于 Hopter 的健壮性特性，控制平面中的错误不会影响数据平面的运行。

网关使用一个开源HAL库[29]，包含360行Rust代码，其中只有8行不安全的代码用于配置外设。编译后的二进制文件需要60.0 KiB的闪存和大约1.3 KiB的SRAM（不包括堆栈）。相比之下，基于FreeRTOS的实现需要21.0 KiB的闪存和大约3.0 KiB的SRAM。

Hopter将数据平面连接与控制平面致命错误隔离开来。为了触发错误，我们让控制台任务在接收命令时跳过检查缓冲区的剩余空闲大小。一个长命令会使缓冲区溢出，导致控制台任务panic并重启。同时，数据平面继续转发数据包，不受干扰。数据包转发延迟不受panic恢复过程的影响。相比之下，相同的缓冲区溢出错误会静默地损坏数据或导致基于FreeRTOS的系统崩溃。由于缺少MPU，Tock不支持此板。

第 7 章 相关工作

使用Rust实现内存安全：与Hopter使用Rust类似，其他操作系统也使用了Rust在内核和应用程序中实现内存安全。最近两项针对个人电脑和服务器的研究，Theseus[8]和RedLeaf[50]，使用Rust实现内存安全，但它们仍然依赖内存管理单元（MMU）放置保护页来防止栈溢出。嵌入式操作系统RIOT[5]、Embassy[11]和RTIC[12]，以及Tock[41]的内核空间，在微控制器上使用Rust实现内存安全。RIOT支持用Rust编写的线程化任务，但同样使用MPU/PMP设置保护区域来防止栈溢出。正如§4.1.2中所讨论的，这种方法不允许从栈溢出中恢复。Embassy、RTIC和Tock的内核空间使用放置在SRAM区域下边界的单一向下增长的堆栈来防止栈溢出破坏内存。然而，它们只支持受限的调度模式，并且无法从栈溢出中恢复。相比之下，Hopter仅通过软件实现内存安全，支持线程化任务的任意调度模式，并且可以从栈溢出中恢复。

致命错误恢复：很少有适用于微控制器的嵌入式操作系统提供与Hopter从致命错误中恢复能力相关的恢复机制。Tock[41]可以自动重启因非法内存访问而失败的任务。Zephyr[27]也可以在此类错误时终止任务，但要求应用程序单独注册错误处理程序。两者都不支持自动清理以实现优雅的任务终止。相比之下，Hopter运行应用程序清理逻辑（析构函数），使系统能够避免后续错误，如死锁。

零延迟中断处理：Hopter使用软锁进行零延迟中断处理，与之前的嵌入式操作系统如eCos[47]、PEASE[58]、PURE[57]和SMX[48]不同，后者将中断处理分为上半部和下半部以避免操作系统中断屏蔽。这种分割设计需要用于延迟操作（下半部）的全局序列化队列，这些操作引用了操作系统对象，因此需要不安全的Rust，因为编译器无法验证引用的生命周期。此外，Hopter的软锁可以通过仅在发生争用时（很少见）推迟操作来提高性能，从而主要通过快速路径减少CPU负载。

结　论

本文描述了Hopter，一个为微控制器设计的基于Rust的嵌入式操作系统。Hopter为应用程序提供内存安全、系统健壮性和高中断响应能力，同时仅需应用最小程度的适配。Hopter通过在使用编译时代码插桩和运行时操作系统支持的FS-语义下执行Rust代码，来防止栈溢出并将此类错误转换为Rust panic。通过将所有致命错误统一为panic，Hopter执行栈展开以从失败的应用程序任务或中断处理程序中回收资源，并且可以自动重启失败的任务。为了确保对中断的及时响应，Hopter引入了一种称为软锁的新颖机制来实现零延迟中断处理。Hopter非常适合资源受限的微控制器，并支持实时工作负载的错误恢复。从栈溢出中恢复的机制也适用于在服务器中运行的进程，软锁在降低响应进程信号的延迟方面也可能有用。

参考文献

[1] Amazon.com, Inc. 2020. FreeRTOS: Real-time operating system for microcontrollers and small microprocessors. https://www.freertos.org. Version 10.3.1.Accessed: 2025-04-24.
[2] Arm Limited. 2021. ARMv7-M Architecture Reference Manual (Issue E.e). https://developer.arm.com/documentation/ddi0403/ee/ Chapter B3.5: Protected Memory System Architecture, PMSAv7. Accessed: 2025-04-24.
[3] Arm Ltd. 2023. Arm Compiler Version 6.6.5 armclang Reference Guide. https://developer.arm.com/documentation/dui0774/l/ Section 2.28. Accessed: 2025-04-24.
[4] AWS open source. 2024. FreeRTOS Documentation. https://www.freertos.org/Documentation/00-Overview Chapter: FreeRTOS Stack Usage and Stack Overflow Checking. Accessed: 2025-04-24.
[5] Emmanuel Baccelli, Cenk Gündoğan, Oliver Hahm, Peter Kietzmann, Martine SLenders, Hauke Petersen, Kaspar Schleiser, Thomas C Schmidt, and Matthias Wählisch. 2018. RIOT: An open source operating system for low-end embedded devices in the IoT. IEEE Internet of Things Journal (2018).
[6] Bitcraze. 2020. Crazyflie: A flying open development platform. https://www.bitcraze.io/.
[7] Bitcraze. 2024. Syslink protocol version 2024.10.https://www.bitcraze.io/documentation/repository/crazyflie2-nrf-firmware/2024.10/protocols/syslink/ Accessed: 2025-04-24.
[8] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. 2020. Theseus: anexperiment in operating system structure and state management. In Proc. USENIXOSDI.
[9] Bosch. 2024. BMI088. https://github.com/BoschSensortec/BMI08x-Sensor-API.Version 1.9.0. Accessed: 2025-04-24.
[10] Ching-Han Chen, Ming-Yi Lin, and Chung-Chi Liu. 2018. Edge computinggateway of the industrial internet of things using multiple collaborative microcontrollers. IEEE Network (2018).
[11] Embassy Project Contributors. 2023. Embassy: The next-generation framework for embedded applications. https://embassy.dev. Accessed: 2025-04-24.
[12] RTIC Contributors. 2023. RTIC: The hardware accelerated Rust RTOS. https://rtic.rs/2/book/en/. Accessed: 2025-04-24.
[13] Jonathan Corbet. 2022. A first look at Rust in the 6.1 kernel. https://lwn.net/Articles/910762/. Linux Weekly News (10 2022).
[14] Microsoft Corporation. [n.d.]. Windows Drivers in Rust. https://github.com/microsoft/windows-drivers-rs. Accessed: 2025-04-24.
[15] MITRE Corporation. 2019. CVE-2019-25001: Flaw in CBOR deserializer allows stack overflow. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-25001. Accessed: 2025-04-24.
[16] MITRE Corporation. 2020. CVE-2020-35858: Parsing a specially crafted message can result in a stack overflow. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-35858. Accessed: 2025-04-24.
[17] MITRE Corporation. 2021. CVE-2021-31572: FreeRTOS before 10.4.3 has an integer overflow for a stream buffer. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-31572. Accessed: 2025-04-24.

[18] MITRE Corporation. 2021. CVE-2021-32020: FreeRTOS before 10.4.3 has insufficient bounds checking during management of heap memory. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-32020. Accessed: 2025-04-24.
[19] MITRECorporation. 2024. CVE-2024-36760: A stack overflow vulnerability found in version 1.18.0 of rhai. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-36760. Accessed: 2025-04-24.
[20] Alex Crichton, Jeehoon Kang, Aaron Turon, and Taiki Endo. 2024. Tools for concurrent programming. https://crates.io/crates/crossbeam. Version 0.8.4. Accessed: 2025-04-24.
[21] Amanieu d'Antras. 2024. Intrusive collections for Rust (linked list and red-black tree). https://crates.io/crates/intrusive-collections. Version 0.9.7. Accessed: 2025-04-24.
[22] The Rust Security Advisory Database. 2018. RUSTSEC-2018-0005: Uncontrolled recursion leads to abort in deserialization. https://rustsec.org/advisories/RUSTSEC-2018-0005.html. Accessed: 2025-04-24.
[23] TheRustSecurityAdvisoryDatabase. 2023. RUSTSEC-2023-0080: Buffer overflow due to integer overflow in transpose. https://rustsec.org/advisories/RUSTSEC-2023-0080.html. Accessed: 2025-04-24.
[24] The Rust Security Advisory Database. 2024. RUSTSEC-2024-0012: Stack overflow during recursive JSON parsing. https://rustsec.org/advisories/RUSTSEC-2024-0012.html. Accessed: 2025-04-24.
[25] Hopter Developers. 2024. Hopter Embedded OS. https://github.com/hopter-project/hopter. Accessed: 2025-04-24.
[26] Redox Developers. 2015. Redox- Your Next(gen) OS. https://www.redox-os.org/. Accessed: 2025-04-24.
[27] Zephyr Developers. 2016. Zephyr Project. https://github.com/zephyrproject-rtos/zephyr. Accessed: 2025-04-24.
[28] Guo Dong, Wang Hongpei, Gao Song, and Wang Jing. 2011. Study on adaptive front-lighting system of automobile based on microcontroller. In Proc. IEEE Int.Conf. Transportation, Mechanical, and Electrical Engineering (TMEE).
[29] Daniel Egger. 2021. Peripheral access API for STM32F0 series microcontrollers. https://crates.io/crates/stm32f0xx-hal. Version 0.18.0. Accessed: 2025-04-24.
[30] Daniel Egger. 2024. Peripheral access API for STM32F4 series microcontrollers. https://crates.io/crates/stm32f4xx-hal. Version 0.22.1. Accessed: 2025-04-24.
[31] Bill Fleming. 2011. Microcontroller units in automobiles [automotive electronics]. IEEE Vehicular Technology Magazine (2011).
[32] Free Software Foundation, Inc. 2023. GCC 15.0.0 Manual. https://gcc.gnu.org/onlinedocs/gcc/ Section 3.12, Program Instrumentation Options. Accessed: 2025-04-24.
[33] Adam Greig. 2023. Low level access to Cortex-M processors. https://crates.io/crates/cortex-m. Version 0.7.7. Accessed: 2025-04-24.
[34] Mehedi Hasan, Maruf Hossain Anik, and Sharnali Islam. 2018. Microcontroller based smart home system with enhanced appliance switching capacity. In Proc.IEEE HCT Information Technology Trends (ITT).

[35] Robert Hieb, R Kent Dybvig, and Carl Bruggeman. 1990. Representing control in the presence of first-class continuations. ACM SIGPLAN Notices (1990).
[36] Free Software Foundation Inc. 2023. Exception handling and frame unwind runtime interface routines.
https://github.com/gcc-mirror/gcc/blob/721cdcd1ddde0738deb08895e113a8db84187a14/libgcc/unwind.inc. Accessed: 2025-04-24.
[37] FortuneBusiness Insights. 2024. Embedded Systems Market Size, Share & COVID-19 Impact Analysis, By Component, By Type, By Application, and Regional Forecast, 2023-2030.
https://www.fortunebusinessinsights.com/embedded-systems-market-108767. Accessed: 2025-04-24.
[38] Future Market Insights. 2024. Embedded System Market Analysis from 2024 to 2034.
https://www.futuremarketinsights.com/reports/embedded-system-market. Accessed: 2025-04-24.
[39] Global Market Insights. 2024. Embedded Systems Market Size- By Component, By Application, By Function & Forecast, 2024- 2032.
https://www.gminsights.com/industry-analysis/embedded-system-market. Accessed: 2025-04-24.
[40] Prateek Khurana, Rajat Arora, and Manoj Kr Khurana. 2014. Microcontroller based implementation of Electronic Stability Control for automobiles. In Proc.IEEE Int. Conf. Advances in Engineering & Technology Research.
[41] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. 2017. Multiprogramming a 64kb computer safely and efficiently. In Proc. ACM SOSP.
[42] LLVM. 2025. AttributorAttributes.cpp- Attributes for Attributor deduction.
https://llvm.org/doxygen/AttributorAttributes_8cpp_source.html. Accessed: 2025-04-24.
[43] LLVM. 2025. FunctionAttrs.cpp- Pass which marks functions attributes.
https://llvm.org/doxygen/FunctionAttrs_8cpp_source.html. Accessed: 2025-04-24.
[44] Zhiyao Ma, Guojun Chen, and Lin Zhong. 2023. Panic Recovery in Rust-based Embedded Systems. In Proc. ACM PLOS.
[45] Zhiyao Ma and Lin Zhong. 2023. Bringing Segmented Stacks to Embedded Systems. In Proc. ACM HotMobile.
[46] Aleksander Malinowski and Hao Yu. 2011. Comparison of embedded system design for industrial applications. IEEE Trans. Industrial Informatics (2011).
[47] Anthony J Massa. 2002. Embedded software development with eCos. Prentice Hall Professional.
[48] Ralph Moore. 2005. Link Service Routines. Micro Digital (2005).
[49] Mohamed Abd El-Latif Mowad, Ahmed Fathy, Ahmed Hafez, et al. 2014. Smart home automated control system using android application and microcontroller. Int. Journal of Scientific & Engineering Research (2014).
[50] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. 2020. RedLeaf: Isolation and Communication in a Safe Operating System. In Proc. USENIX OSDI.
[51] Pimoroni. 2024. PMW3901. https://github.com/pimoroni/pmw3901-python/tree/main. Version 1.0.0. Accessed: 2025-04-24.
[52] Pololu. 2022. Vl53l1x. https://github.com/pololu/vl53l1x-arduino. Version 1.3.1. Accessed: 2025-04-24.

[53] LLVM Project. 2023. Handling Level 1. Implementation of C++ ABI Exception https://github.com/llvm/llvm-project/blob/f42482def236999b0f7896c09cd714b708861c8b/libunwind/src/UnwindLevel1.c. Accessed: 2025-04-24.
[54] RISC-V. 2024. The RISC-V Instruction Set Manual: Volume II (Version 20240411). https://riscv.org/technical/specifications/ Chapter 3.7: Physical Memory Protection. Accessed: 2025-04-24.
[55] Rust on Embedded Devices Working Group et al. [n.d.]. The Embedded Rust Book. https://docs.rust-embedded.org/book/. Accessed: 2025-04-24.
[56] Alice Ryhl. 2023. Rewriting Binder Driver in Rust. https://lore.kernel.org/rust-for-linux/20231101-rust-binder-v1-0-08ba9197f637@google.com/. Accessed: 2025-04-24.
[57] Friedrich Schon, Wolfgang Schroder-Preikschat, Olaf Spinczyk, and Ute Spinczyk. 2000. On interrupt-transparent synchronization in an embedded object-oriented operating system. In Proc. IEEE Int. Symp. Object-Oriented Real-Time Distributed Computing (ISORC).
[58] Wolfgang Schröder-Preikschat. 1994. The logical design of parallel operating systems. Prentice-Hall, Inc.
[59] STMicroelectronics. 2020. RM0402 (Rev.6) Reference manual for STM32F412 advanced Arm-based 32-bit MCUs. Chapter 3.4.2: Adaptive real-time memory accelerator (ART Accelerator).
[60] STMicroelectronics. 2024. RM0090 (Rev.21) Reference manual for STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs. https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf Chapter 30.3.3: Receiver-OverrunError. Accessed: 2025-04-24.
[61] STMicroelectronics. 2024. STM32CubeF0 HAL driver MCU component. https://github.com/STMicroelectronics/STM32CubeF0/releases/tag/v1.11.5. Version 1.11.5. Accessed: 2025-04-24.
[62] STMicroelectronics. 2024. STM32CubeF4 HAL driver MCU component. https://github.com/STMicroelectronics/stm32f4xx_hal_driver/releases/tag/v1.8.3. Version 1.8.3. Accessed: 2025-04-24.
[63] Rust Tools Team. 2021. svd2rust. https://github.com/rust-embedded/svd2rust
[64] Theseus Developers. 2023. Support for unwinding the call stack and cleaning up stack frames. https://github.com/theseus-os/Theseus/blob/562a39cf6c662738f7718473f6bbc010970dce53/kernel/unwind/src/lib.rs. Accessed: 2025-04-24.
[65] Vishakha D Vaidya and Pinki Vishwakarma. 2018. A comparative analysis on smart home system to control, monitor and secure home, based on technologies like gsm, iot, bluetooth and pic microcontroller with zigbee modulation. In Proc.IEEE Int. Conf. Smart City and Emerging Technology (ICSCET).
[66] Mathijs van de Nes and Joshua Barretto. 2023. Spin-based synchronization primitives. https://crates.io/crates/spin. Version 0.9.8. Accessed: 2025-04-24.
[67] Rob Von Behren, Jeremy Condit, Feng Zhou, George C Necula, and Eric Brewer. 2003. Capriccio: scalable threads for internet services. ACM SIGOPS Operating Systems Review (2003)

北京理工大学本科生毕业设计（论文）外文翻译

指导教师意见：选择的论文与毕业设计密切相关。翻译内容基本准确，能完整表达原文含义。

指导教师签字： 

日期：2026 年 5 月 26 日